

# D3: Enhancing Reconfigurable Datacenters with Adaptive Demand-Oblivious and Demand-Aware Integration

Johannes Zerwas<sup>a</sup>, Chen Griner<sup>b</sup>, Stefan Schmid<sup>c</sup>, Chen Avin<sup>b</sup>

<sup>a</sup>*TUM School of Computation, Information and Technology, Technical University of Munich, Germany*

<sup>b</sup>*School of Electrical and Computer Engineering, Ben Gurion University of the Negev, Israel*

<sup>c</sup>*TU Berlin & Fraunhofer SIT, Germany*

---

## Abstract

The explosively growing communication traffic in datacenters imposes stringent performance requirements on the underlying networks. Over the last years, researchers have developed innovative optical switching technologies that enable reconfigurable datacenter networks (RDCNs) with fast topology reconfigurations. This paper presents D3, a novel RDCN architecture suited for PoD-level deployment, improving throughput and flow completion time and enabling scalable composition within hyperscale datacenter networks. D3 adapts its links and packet scheduling toward the evolving demand, combining both demand-oblivious and demand-aware behaviors. D3 relies on a decentralized network control-plane supporting greedy, integrated-multihop, IP-based routing, allowing to react quickly and locally to topological changes without overheads. Moreover, we argue that D3 can be implemented using recently proposed optical-switching architectures. Using extensive evaluations, we find that D3 improves throughput by up to 15% and preserves competitive flow completion times compared to the state-of-the-art. We further provide an analytical explanation of the superiority of D3, introducing an extension of the well-known Birkhoff-von Neumann decomposition, which may be of independent interest.

**Note:** This is the author's accepted manuscript of an article published in *Computer Networks* (Elsevier, 2025).

The final version is available at: [doi.org/10.1016/j.comnet.2026.112255](https://doi.org/10.1016/j.comnet.2026.112255)

© 2026. This manuscript version is made available under the CC-BY-NC-ND 4.0 license: [Creativecommons.org/licenses/by-nc-nd/4.0](https://creativecommons.org/licenses/by-nc-nd/4.0)

**Keywords:** Datacenter Networks, Optical networks, Network architecture, Topology Engineering

---

## 1. Introduction

Communication traffic in datacenters is growing explosively. This is due to the popularity of data-centric cloud applications such as batch processing and distributed machine learning (ML), and the trend towards resource disaggregation in datacenters [47, 38]. This results in increasingly stringent performance requirements on the underlying datacenter networks (DCNs), which are reaching their capacity limits. Accordingly, over the last few years, researchers have made great efforts to improve the capacity of DCNs [2, 60, 43, 25, 71, 34, 61].

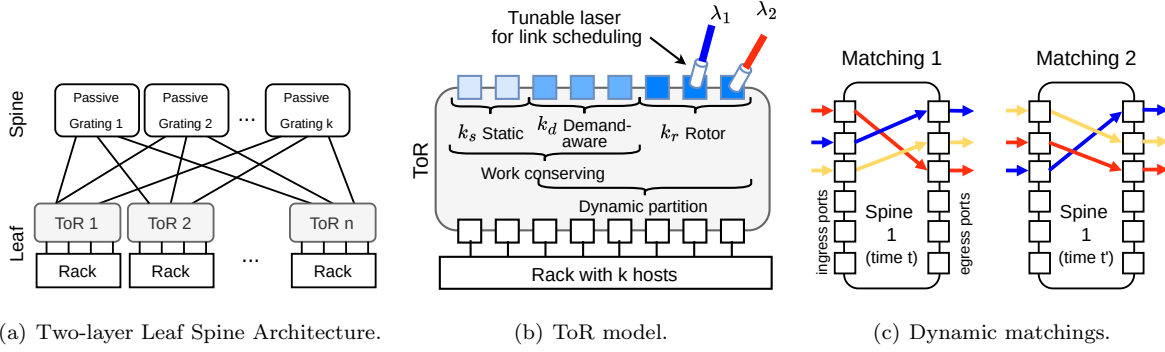


Figure 1: Overview of D3’s architecture. (a)  $n$  ToRs connect in a two-layer leaf-spine topology to  $k$  passive gratings. Solid lines indicate bidirectional links. (b) ToR model: a tuneable laser at the transceivers can adjust the wavelength to select an egress port. There are three links (ports) scheduler classes STATIC, DEMAND-AWARE, and ROTOR. (c) By reconfiguring the wavelength at the ToR, the matching (circuits) between the Input/Output ports at the spine (passive grating) can change dynamically.

Emerging optical circuit switching (OCS) technologies offer several advantages over traditional datacenters built entirely using electrical packet switches (EPS). A key innovative capability that this paper will focus on enabled by OCS is the possibility of making datacenter networks dynamic and *reconfigurable* [26]: such switches can be used to quickly change the networks’ topology. The process of modifying the datacenter topology, is referred to as **Topology Engineering (ToE)** [75, 12, 64, 23]. In principle, ToE in reconfigurable datacenter networks (RDCNs) enable better use of the network capacity by providing topological shortcuts, hence reducing multi-hop forwarding and saving *bandwidth tax* at the cost of *latency tax* [24]. Indeed, it has been shown that even *demand-oblivious* RDCNs such as RotorNet [46], Opera [44], Sirius [7], Mars [1], and Shale [4], whose topology changes periodically, can significantly improve throughput in datacenters. *Demand-aware* RDCNs even allow to adapt the topology to account for structure in the workload [5, 8, 54, 33, 48, 80, 76], and, e.g., optimize the rack-to-rack interconnect toward elephant flows. Demand-aware networks such as ProjecToR [21], Gemini [75], or Cerberus [24], among others [78, 32, 27, 14, 28, 18, 62, 15, 67, 20, 51, 69, 6, 57, 58, 17, 35, 9, 52, 39, 11], successfully exploit the spatial and temporal locality in traffic patterns to improve performance, even if reconfigurations are performed infrequently [75, 52].

The research community is still advancing both approaches to ToE in RDCNs, e.g. [4, 39]. Accordingly, the best (dynamic) topology depends on the traffic and applications it needs to serve [24]. For example, shuffling traffic of map-reduce applications, or all-gather in ML, will benefit from demand-oblivious RDCNs, based on optical *rotor* switches [46, 44, 7], due to the all-to-all nature of the traffic. At the same time, workloads that are skewed, like reduce-based operations in ML or applications with large flow sizes like in datamining [22] are best served on demand-aware RDCNs, using demand-aware optical switches [75, 14, 18]. Latency-critical mice flows, on the other hand, are best served on static topologies such as fat-trees or

| Concept (Approach)                          | Adopted from    | D3 improvement         |
|---------------------------------------------|-----------------|------------------------|
| Demand-aware links (Distributed matching)   | ProjecToR [21]  | Non-segregated routing |
| Rotor scheduling (RotorLB)                  | RotorNet [46]   | Local LB               |
| ToR tunable lasers (Passive gratings)       | Sirius [7]      | Demand-aware ports     |
| Three sub-topologies (Static partition)     | Cerberus [24]   | Dynamic partition      |
| Greedy routing (de Bruijn topology)         | Duo [74]        | Rotor ports            |
| Time division Rotor and Demand-aware        | NegotiaToR [39] | Static ports           |
| Birkhoff-von Neumann (Greedy decomposition) | Eclipse [67]    | Mixed decomposition    |

Table 1: D3 combines and extends several important concepts from existing systems.

Xpander [2, 60, 43, 34], which do not have a latency tax resulting from reconfiguration delays. Matching traffic patterns to the right topology designs is, hence, critical for performance [24].

While communication patterns in datacenters naturally change over time (e.g., due to increasing loads or evolving applications), it can be difficult in practice to tailor datacenter network topology toward its demand.

Our proposed system design, D3, supports both independent medium-scale datacenter deployments, based on a two-layer leaf-spine topology with OCS at the spine, and hyperscale datacenter architectures, where it serves as a point of delivery (PoD). D3 divides the network into adaptive sub-topologies that enable addressing such an evolution in a practical manner by dynamically splitting its infrastructure between different modes of operations. Our paper is motivated by recent optical switching technology introduced in Sirius [7], which enables us to perform ToE using Top-of-the-Rack (ToR) switches. Sirius is a leaf-spine architecture that uses nanosecond tunable lasers at the leaves (ToRs) and passive wavelength gratings (AWGR) that route light based on wavelengths at its spine switches. To change the topology, the optical circuits that connect the ToRs are reconfigured via the lasers. ToE in our case boils down to what we call *link scheduling*, scheduling the wavelengths that each laser uses at any time. Sirius realizes its RDCN using demand-oblivious link scheduling. However, as the authors of Sirius already observed, the optical switches can also be used for adaptive link scheduling in a demand-aware manner. In fact, we claim that the switch can be used in a polymorphic manner, allowing for quick changes between demand-oblivious, demand-aware, or static link scheduling. Changing the optical topology, whether demand-oblivious or demand-aware, simply means changing the type of scheduler that controls the tunable lasers. Moreover, different lasers (ports), even at the same ToR, can use different types of link schedulers. This, in principle, enables fully dynamic, *self-adjusting* datacenter networks: networks whose topology can be instantaneously optimized towards the traffic at any time.

**Contribution.** This paper explores how to design such flexible self-adjusting datacenter networks using ToE. In particular, we propose a novel and *feasible* datacenter or PoD architecture, D3, based on the Sirius [7] architecture. But, in contrast to Sirius, D3 topology consists of three different sub-topology components

(namely a static, a dynamic demand-oblivious, and a dynamic demand-aware sub-topology) that can be adjusted quickly toward the evolving traffic. Figure 1 presents D3’s core architecture, we explain it in more detail in Section 3 and discuss scaling beyond the two-layer design in Section 3.8. We further study how to support such flexible architectures using efficient and *jointly* optimized link scheduling (to realize the dynamic topology) and packet scheduling (the packet forwarding strategy), as well as network and transport layers. To this end, D3 relies on a decentralized network control plane supporting greedy integrated-multihop, IP-based routing, which allows to quickly and locally react to topological changes without overhead and with minimal packet reordering. D3 comes with theoretical underpinnings, and we explain D3’s superiority analytically. That is, we prove that a mixture of demand-aware and demand-oblivious scheduling can improve the *demand completion time* of a scheduling that uses only a single type. For our analysis, we contribute a novel extension of the Birkhoff–von Neumann (BvN) matrix decomposition, which supports mixed topologies and may be of independent interest. Results from extensive packet-level simulations with realistic and synthetic workloads illustrate that D3 improves throughput by up to 15% while preserving or even improving flow completion times compared to the state-of-the-art. We present the main concepts within the paper and defer some technical details to the Appendix.

**Putting things into perspective and related work.** To achieve its goals, the design and implementation of D3 stand on the shoulders of giants by combining and extending several important concepts and approaches from existing systems. Table 1 summarizes the main points. Similar to systems such as ProjectoR [21], D3 leverages demand-aware links that are scheduled in a decentralized manner, using distributed matching algorithms; however, D3 also supports *non-segregated routing*, allowing an improved utilization of the available network resources. Similar to systems such as RotorNet [46], D3 leverages fast rotor scheduling to quickly shuffle some of its traffic in a demand-oblivious manner; however, in contrast to the RotorLB approach in prior work, D3 employs a *local approach* named LocalLB, which significantly reduces control plane overheads and does not require global synchronization. As mentioned, we use Sirius [7] as the infrastructure for D3, and although the Sirius technology makes D3 feasible, the two systems are conceptually different. First and foremost, Sirius is demand-oblivious as a philosophy that trades throughput with simplicity. To achieve higher throughput, D3 requires a more complex, non-trivial control mechanism to enable *demand-aware topologies and multi-hop packet forwarding*. Like Cerberus [24], D3 is tailored toward the traffic mix it serves, using a network topology that matches the demand; however, while in the Cerberus topology, the partition into static, demand-oblivious, and demand-aware dynamic switches is *fixed* by different switch types, D3 allows for a *dynamic port partitioning* of the topology to sub-topologies. This ability leverages flexible link scheduling (enabled by Sirius technology). Moreover, while Cerberus presented a more conceptual contribution and a flow-based simulation, D3, presents a *feasible implementation* and a packet-level simulation, as well as transport layer solutions. D3 further builds upon concepts from Duo [74]; in particular, it sup-

ports greedy routing by relying on an enhanced de Bruijn topology; however, D3 additionally also supports *rotor ports* and demand-oblivious sub-topology to shuffle traffic quickly. NegotiaToR [39], is based on the Sirius infrastructure, and uses both demand-oblivious and demand-aware scheduling. However, there exists a time division of the two scheduling types, and they do not exist together, unlike in D3. More importantly, NegotiaToR does not implement any static ports, which are a significant part of the functionality of D3, particularly for delay-sensitive flows, and its demand-oblivious scheduling is not typically used to send any significant bulk of the data. At its core, D3 relies on a greedy Birkhoff-von Neumann (BvN) decomposition algorithm, as also used in Eclipse [67]. For D3, we extend this BvN algorithm and introduce a *novel matrix decomposition* that supports mixed topology types.

We note that the topic of RDCN design and implementation is very active, with recent research papers at the top conferences [4, 39, 45, 37]. To the best of our knowledge, D3 presents a unique perspective and contribution.

**Organization:** The remainder of this paper is organized as follows. We motivate the benefits of dynamic, oblivious, and demand-aware link scheduling in Section 2. Section 3 describes the architecture and design of D3. We report on our results from extensive packet-level simulations in Section 4 and provide theoretical justification for the superiority of D3 over OPERA and DUO in Section 5. We conclude our contribution in Section 6. The paper also contains supplementary material starting at [Appendix A](#).

## 2. Motivation for Links Scheduling

Our work is motivated by the observation that, in order to maximize throughput, the datacenter topology must adjust to the specific traffic pattern it serves [24].

We illustrate this with a simple motivating example. Consider an RDCN with five hosts and five racks (single host per rack). In this example, a *single reconfigurable port* in each ToR switch provides a single, ToR-to-ToR, directed matching (the reconfigurable topology) between the racks (and hosts). The link scheduler determines the current matching (topology) to use. The throughput-maximizing topology resp. link scheduling depends on the  $(5 \times 5)$  host-to-host demand matrix.

Figure 2 presents three different demand matrices, all with the same amount of total data to transmit: (a) a sparse demand matrix that forms a permutation, (b) a uniform demand matrix where all host pairs have equal demand, and (c) a mixed scenario where darker entries denote more demand. The maximum throughput is measured as the minimum *demand completion times* (DCT) it takes to send all the demand in each matrix. For each demand, this minimum is achieved via a different links schedule. Next to each matrix, we show its optimal schedules. The time axis shows two units of time (e.g., 100ms each), and we indicate the time it takes to serve the given demand matrix (the demand-completion time). On the time axis, in blue color, we show the circuit-hold times (i.e., the times that circuits can be used to transmit), and the gaps between

them represent the reconfiguration times between two consecutive matchings. Recall that reconfiguration times are typically shorter for demand-oblivious link scheduling (which does not involve any optimization and uses predefined matchings) than for demand-aware link scheduling that needs to make decisions, like what links to establish. Figure 2(a): A permutation matrix is best served by a single demand-aware matching that provides direct connectivity between the racks that need to communicate. It requires a single link change (that takes longer) but then makes no more changes. Figure 2(b): A uniform demand matrix is best served by a fast and oblivious (periodically) changing sequence of matchings (i.e., a rotor switch), always providing one direct connection between each rack pair during a periodic cycle. Figure 2(c): For a mixed demand, a combination of scheduling gives the best results: a demand-aware link scheduling configures a matching for the first unit of time, and then a sequence of fast predefined oblivious matchings is used in the second unit of time.

We conclude that different types of link schedulers (which have given, but different, reconfiguration times) are needed to maximize throughput. Moreover, the *type* of the link scheduler on a port may change over time. However, realizing such flexible datacenter networks is challenging and requires efficient and practical links and packet schedulers, routing algorithms, and a transport protocol that can support a high degree of dynamics. In the next section, we describe the design of D3 that achieves these goals.

### 3. The System Design of D3

This section introduces the system design of D3. We first present the general architecture and topology components.

Then, we describe in more detail the de

Brujn-based demand-aware sub-topology and the packet scheduler for the demand-oblivious sub-topology. We show how integrated forwarding can be realized in practice and describe the transport layer of D3. Lastly, we discuss how D3 can be built on top of Sirius and the implications for cost-effectiveness.

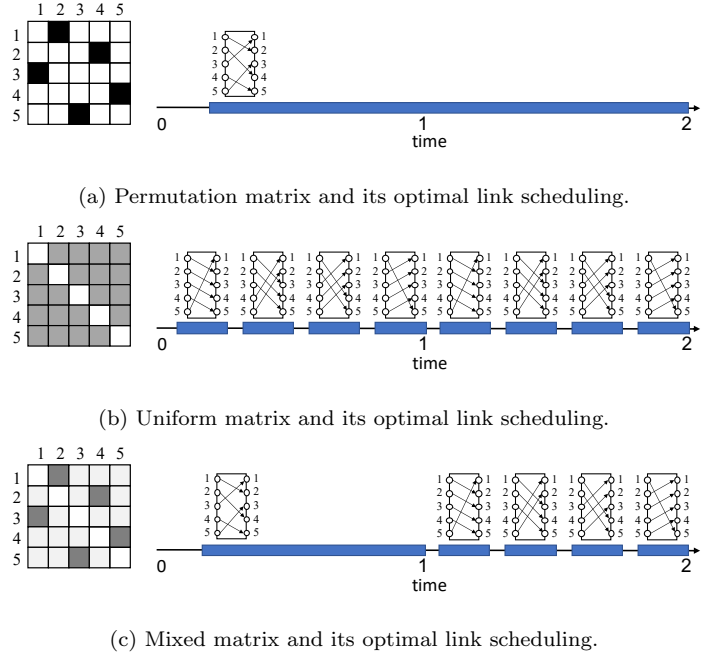


Figure 2: Motivation for D3: the optimal dynamic topology resp. link scheduling depends on the demand matrix, which may change over time.

### 3.1. Basic Architecture and Link Scheduler

Figure 1 overviews D3’s architecture. D3 relies on a two-layer, leaf-spine optical topology that can be described with the ToR-Matching-ToR (TMT) model [24] (Figure 1(a)). The leaf layer consists of  $n$  ToR switches  $T_i$  ( $1 \leq i \leq n$ ). Each ToR has  $k$  up- and  $k$  downlink ports of rate  $r$ . The latter ones connect to end-hosts so that the network contains  $h = n \cdot k$  hosts in total. Since the uplinks are bidirectional, we can further separate them into  $k$  unidirectional ingress and  $k$  egress ports. The spine layer consists of  $k$  passive gratings<sup>1</sup> connected to the ports at the ToRs. Each uplink port in a ToR has a *wavelength-adjustable laser* (Figure 1(b)). The wavelength determines the egress port of the grating (i.e., spine switch), cf. Figure 1(c). Changing the wavelength reconfigures a new optical circuit (topological link), as proposed and demonstrated in Sirius [7]. Properly configuring the laser wavelengths at each time slot creates a directed multi-hop topology that is based on a set of  $k$ -directed matchings between the ToRs, i.e., the  $i$ -th ports in the ToRs are connected to the  $i$ -th grating, creating the  $i$ -th matching (of size  $n$ ). This is exactly the structure of the TMT model.

In turn, the  $k$  matchings (and the  $k$  ports in each ToR) are partitioned into three *link scheduling* classes:<sup>2</sup> STATIC, ROTOR, and DEMAND-AWARE. The sizes of the classes are  $k_s, k_r, k_d$  respectively, keeping the constraint  $k = k_s + k_r + k_d$ . A partition to three classes is implemented by assigning the  $k$  ports in each ToR switch to the different link schedulers (STATIC, ROTOR, and DEMAND-AWARE) in a consistent and symmetric way. In all ToRs, the same  $k_s$  ports use the STATIC link scheduler, the same  $k_r$  ports use the ROTOR link scheduler, and the same  $k_d$  ports use the DEMAND-AWARE link scheduler.

An important feature of D3, as we discuss later, is that the sizes of  $k_s, k_r$  and  $k_d$  can change over time as long as they keep the constraint. Each of the three link scheduling classes creates a different sub-topology that is described in the following:

**Static sub-topology (Static):** The  $k_s$  static ports do not reconfigure the links over time. The resulting topology can be described as the union of  $k_s$  static matchings. The static ports provide basic connectivity between the ToRs and can be used to create  $k_s$ -regular graphs, such as expander graphs, providing low latency, multi-hop routing for short flows, and control messages. Specifically, D3 relies on de Bruijn graphs [36], as described later.

**Demand-aware sub-topology (Demand-aware):** The demand-aware topology consists of a collection of  $k_d$  DEMAND-AWARE reconfigurable ports per ToR. They can be flexibly reconfigured to *any* possible  $k_d$ -regular directed graph between the ToRs and change it over time. For example, these ports can be used to create direct connections between ToR pairs with high communication demand. The pure physical reconfiguration delay of such link is in the order of nanoseconds [7]. However, reconfiguring the DEMAND-

<sup>1</sup>Originally,  $k$  *active* optical spine switches in the TMT model.

<sup>2</sup>Denoted as spine switch types in the TMT model.



AWARE ports requires coordination between the ToRs (e.g., for data collection and decision making). To account for this, we let the system parameter  $R_d$  denote the *reconfiguration delay* of a demand-aware port. We consider the default value  $R_d = 1ms$ , which is assumed to be realistic under current technologies, in particular, if decisions are made locally between pairs of ToRs. The practical time may vary and also depends on the traffic, prediction quality, and implementation. The circuit-hold time after each reconfiguration is dynamic (not constant) during the operation of a DEMAND-AWARE port. We require it to be much larger than  $R_d$  to achieve a high duty cycle and efficient operation.

**Rotor-based sub-topology (Rotor):** The rotor-based topology is formed by the set of  $k_r$  ROTOR ports per switch. Each ROTOR port cycles through  $n-1$  predefined matchings, emulating a complete graph. Every ROTOR port cycles through the same  $n-1$  matchings but using a different time shift. This synchronization provides an average *cycle time* of  $\frac{n-1}{k_r}$  slots to complete a single emulation of a complete graph between all ToRs. The symmetry between the ports' link scheduling facilitates a flexible way to enable or disable a ROTOR link scheduler on a port since the only difference between the ROTOR ports is their time shift. The slot time of the ROTOR link scheduling class is defined by a circuit-hold time  $\delta$  plus a reconfiguration time  $R_r$ , which includes the physical link reconfiguration (wavelength change) and additional guard time to empty active links. The *duty cycle*  $\eta$  is the fraction of time traffic can be sent in a slot:  $\eta = \frac{\delta}{\delta + R_r}$ . The slot time is tuneable and depends on the reconfiguration time, where a reasonable setup is to achieve  $\eta > 90\%$  as in [46, 44, 7]. Here, we assume  $\eta \approx 98\%$  as in [44].

Since the partition of sub-topologies is determined by the link scheduling classes of the ports, but not by the underlying infrastructure, the assignment of ports to link scheduling classes can change over time and, by that, change the sub-topologies sizes. In contrast to previous proposals [24], this adds a new dimension of demand-awareness to D3 where  $k_s$ ,  $k_d$ , and  $k_r$  can dynamically adapt. For instance, a DEMAND-AWARE port can become a ROTOR port if the traffic pattern has changed such that the ROTOR sub-topology is under-provisioned. Although this *dynamic partitioning* of the topology can be used for all three classes, D3 limits it to the DEMAND-AWARE and ROTOR ports.

The three sub-topologies have different properties for the packet forwarding behavior: STATIC and DEMAND-AWARE can utilize work-conserving forwarding (in the sense of store and forward), whereas ROTOR requires a *packet scheduling* algorithm to transmit packets successfully. Both parts are introduced in the subsequent sections.

### 3.2. STATIC and DEMAND-AWARE Topology

D3 augments a de Bruijn graph-based static topology built from the STATIC ports with dynamic connections (short-cuts) on the DEMAND-AWARE ports. This concept has recently been shown to be a promising candidate for high-throughput topologies [74]. In contrast to previous proposals [21, 24], it supports *integrated multi-hop* routing across links from both the STATIC and DEMAND-AWARE topology parts, i.e., a



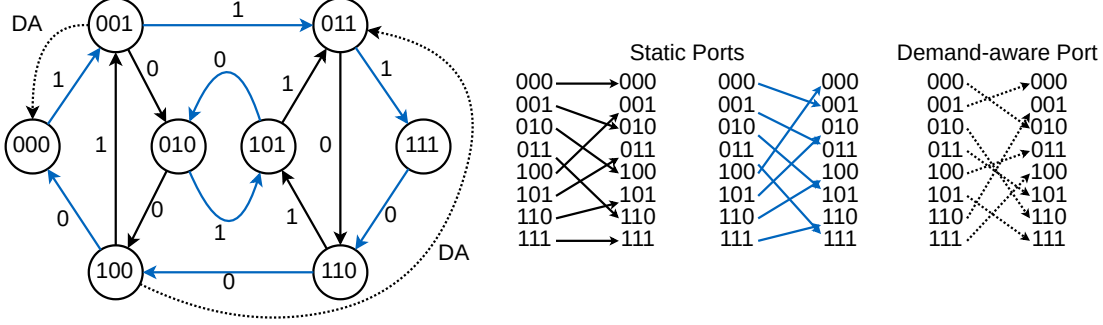


Figure 3: An example of D3’s backbone sub-topology with 8 ToRs: A de Bruijn topology established by two STATIC ports and one DEMAND-AWARE port. Center: two matchings for the STATIC ports, right: the matching of the DEMAND-AWARE ports (for clarity, drawing only two links in the topology).

single packet can traverse both sub-topologies to reach the destination. Moreover, the structural properties of the de Bruijn graph enable (IP-based) *greedy* routing using small forwarding tables [74].

Figure 3 shows an example of a hybrid de Bruijn topology with eight ToRs each having two STATIC and one DEMAND-AWARE ports (Node IDs are in binary representation). Therefore, it is a union of three matchings, two STATIC and demand-oblivious, and one dynamic and DEMAND-AWARE. The matchings are shown in the right half of the figure, and we provide the main details below. Formally, the *static* de Bruijn topology is defined as follows [36]:

**Definition 1** (de Bruijn topology). *For integers  $b, d > 1$ , the  $b$ -ary de Bruijn graph of dimension  $d$ ,  $DB(b, d)$ , is a directed graph  $G = (V, E)$  with  $n = |V| = b^d$  nodes and  $m = |E| = b^{d+1}$  directed edges. The node set  $V$  is defined as  $V = \{v \in [b-1]^d\}$ , i.e.,  $v = (v_1, \dots, v_d), v_i \in [b-1]$ , and the directed edge set  $E$  is:  $\{v, w\} \in E \Leftrightarrow w \in \{(v_2, \dots, v_d, x) : x \in [b-1]\}$  where  $[i] = \{0, 1, \dots, i\}$ .*

The direct neighbors of a node  $v$  are determined by a left *shift* operation on the node’s address and appending a new symbol  $y \in [b-1]$ . In Figure 3 where  $b = 2$  is the binary case, shifting 001 and appending 0 to it results in 010, which is one of the two neighbors of node 001 in the static sub-topology. In particular, it is the neighbor via port 0. The second neighbor is 011, via port 1, which indicates appending 1 after the shift operation. A key property of de Bruijn graph-based topologies is that they can be constructed from  $b$  directed perfect matchings. Moreover, when using longest prefix matching (LPM), the size of the forwarding table on each node has at most  $bd = O(b \log_b n)$  entries [74].

The essence of the recent proposal Duo [74], was to show that augmenting a de Bruijn topology built from  $k_s$  STATIC matchings, with a  $k_d$  DEMAND-AWARE matchings creates a topology that supports integrated, multi-hop, greedy, work-conserving, LPM-based routing with a forwarding table size of  $O((k_s + k_d) \log_{k_s} n)$  and diameter  $d \leq \log_{k_s} n$  (Theorem 3.2 within). Moreover, the update cost of a forwarding table upon a DEMAND-AWARE link change is small and can be performed locally by communicating with the new

|                 | RotorLB (RLB) | LocalLB (LLB)  |
|-----------------|---------------|----------------|
| Offloading      | Not supported | Non-local      |
| Non-local       | FS            | FS             |
| Local           | FS            | FS             |
| Indirect (Sync) | FS (global)   | Greedy (local) |

Table 2: RotorLB Vs. D3’s LocalLB.

neighbor. Routing on the de Bruijn topology can be implemented using standard IP and packet switching equipment [74]. Although Duo has many benefits that we implement in D3, it does not have a ROTOR sub-topology as we discuss next.

### 3.3. Packet Scheduler for ROTOR Topology

As in previous work [7, 46, 44], the highly dynamic ROTOR sub-topology requires a non-work-conserving packet scheduling algorithm to transmit packets successfully. Links between ToRs are constantly and systematically changing, emulating a complete graph between the ToRs while being oblivious to the demand. Therefore, packets potentially need to be stored while waiting for their next-hop link to be reconfigured.

The state-of-the-art approach for this challenging task is to consider both *direct* (single hop) and *indirect* (of at most two hops) routes and use Valiant-based routing [66] (via a random intermediate helper node) to achieve load balancing.<sup>3</sup> This approach was shown to provide high throughput, namely the RotorLB (RLB) scheduling proposed with RotorNet and Opera [7, 44]. Unfortunately, RLB introduces significant control-plane overhead: Since intermediate buffers cannot easily handle overflows, a tight sender-receiver flow control mechanism is necessary, which results in 3-way communication at the beginning of every slot. In contrast, D3 implements a simpler packet scheduler, *LocalLB* (LLB), that does not require such global synchronization. The decisions of what to send indirectly are made (rack-)locally. To alleviate overload of specific racks/hosts, D3 uses the STATIC and DEMAND-AWARE sub-topologies to offload and forward long waiting traffic to its destination. In a nutshell, the offloading mechanism shifts excess non-local traffic to the STATIC and DEMAND-AWARE part if local traffic exceeds a specified threshold.

Table 2 summarizes the scheduling process at the beginning of a slot for RLB vs. LLB. First, with LLB each host checks if non-local traffic demand must be offloaded from the ROTOR to the backbone sub-topology. The offloading follows a simple, greedy logic described in Algorithm 1. Each host  $v$  checks for each destination  $u$  (that is not in the same rack) if the *local* demand exceeds a given threshold  $d_{\text{off}}$ . If so, all non-local traffic to  $u$  is offloaded to the de Bruijn-based backbone. If the local demand towards  $u$  is below

<sup>3</sup>For short, time-sensitive packets, Opera [44] uses longer multi-hop paths routing. Still, these packets capture only a small fraction of the total traffic.

---

**Algorithm 1:** D3 Offloading (from Rotor to de Bruijn) at host  $v$ .

---

**Input:** Local and non-local demand per destination (host)

**Output:** Demand per destination (host) to offload to backbone

```
1 for Destination hosts  $u$  in other racks than  $v$  do
2   if Local demand to  $u$  is  $> d_{\text{off}}$  then
3     Offload all non-local demand to  $u$  to backbone
4   else
5     if Sum of local and non-local demand to  $u$  is  $> d_{\text{off}}$  then
6       Offload non-local demand to  $u$  that exceeds slot capacity left after assigning equal share of the
        slot capacity to the local demand
```

---

the threshold, the algorithm compares the total demand from  $v$  to  $u$  and offloads excess *non-local* demand to the backbone. A possible choice for  $d_{\text{off}}$  is the volume that can be sent in a single slot and a single path, assuming all-to-all traffic and fair-share, i.e.,  $\frac{C}{k}$  (we use this as the default value). Note that LLB offloads only non-local demand to the backbone. The intuition is to free ROTOR capacities for local traffic which is expected to have a more uniform pattern. Moreover, the offload reduces the waiting time of the non-local demand to reach its final destination.

Afterwards, remaining non-local and the local demand are scheduled with the same iterative fair-share over source and destination hosts as in RLB. Next, RLB synchronizes demand information which is again fairly distributed source and destinations on the remaining capacity. In LLB, this step is not necessary since it uses a local greedy share approach combined with the local offloading, saving complexity.

Algorithm 2 presents pseudo-code of LLB. The complete algorithm and notation are presented in Algorithm 4 and Table A.3 in the appendix. The algorithm takes a per-ToR perspective and requires host-level source-destination pair demand information (local and non-local) for all hosts in its rack towards all hosts in other racks. It outputs the amount of traffic per destination host to be sent for all hosts in its rack of the non-local demand, local demand and to send indirectly.

First, the algorithm initializes the data structures with the remaining sending capacity per host in the rack and the remaining receiving capacity for all hosts in the connected racks. Afterwards in lines 2-5, it allocates the second hop non-local traffic and local traffic that can be sent directly to the destination. For each of the two demand types, it iterates over the ROTOR ports and, per port, allocates the demand to hosts in the connected rack using an iterative fair-share algorithm considering the remaining capacities.

The second part (lines 6-17) allocates traffic to be sent via an intermediate host. Again, the algorithm iterates over the ROTOR ports. Per port, it iterates over the hosts in the destination rack (with the order changing in a round-robin fashion between calls of the packet scheduling). For each such candidate interme-

---

**Algorithm 2:** D3 Packet-Scheduling (LocalLB) for ToR  $t$ .

---

**Input:** Per SD-pair (host) local and non-local demand

**Output:** Traffic allocation of non-local demand to be sent in this slot, local demand to be sent directly, and local demand to be sent indirectly.

```
1 Initialize  $C_i^{TX} \leftarrow \bar{C}$  as remaining sending capacity per host and  $C_{i,b}^{RX} \leftarrow \frac{C}{k}$ ,  $\forall i = 1 \dots k, \forall b = 1 \dots k_r$  as per
   Rotor and host remaining receiving capacity
   // Allocate Non-local (N) on the 2nd hop and local (L)
2 for Non-local and local demand separatedly (in this order) do
3   for Every Rotor-link  $u$  do
4     Allocate demand between hosts in  $t$  and hosts in the rack connected via  $u$  with iterative fairshare
       considering the remaining capacities in TX and RX directions
5   Update remaining capacities, remaining demands and allocated demands
   // Allocate new indirect traffic on the remaining capacity
6 for Every Rotor-link  $u$  do
7   for all hosts in ToR connected via  $u$  as  $a$ , order round-robin between calls of this algorithm (over slots) do
8     // Identify candidates for local indirect traffic allocation
9      $y \leftarrow$  Considering only local demand, that exceeds the equal-share capacity, cap the remaining local
       demands by the remaining sending capacity per host  $C^{TX}$  and find the largest value per source host
10     $n_z \leftarrow$  number of such demand  $> 0$ 
11    for every such demand  $z > 0$  in non-decreasing order do
12      if  $z$  exceeds the remaining receiving capacity of  $a$  then
13        Equally allocate remaining receiving capacity of  $a$  across all the found remaining local demands
           to be sent indirectly
14        Update remaining capacities, remaining demands and allocated demands.
15        Break
16      else
17        Allocate  $z$  to be sent indirectly and update remaining capacities, remaining demands and
           allocated demands
18      Decrement  $n_z$ 
```

---

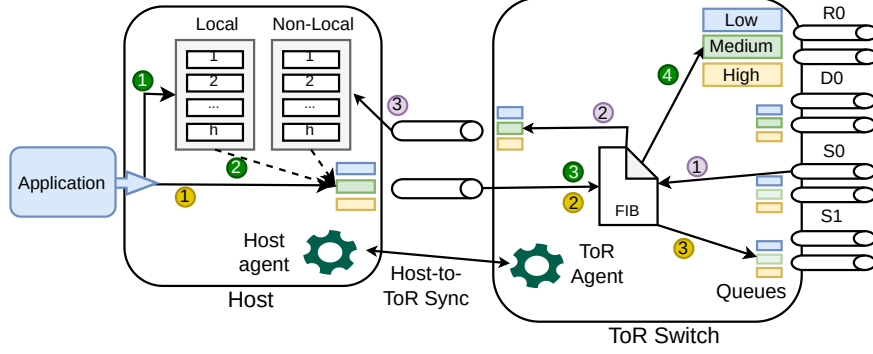


Figure 4: Data- and control plane for hosts and ToRs.

diate host, it searches for the destination with the highest demand per source host while accounting for the remaining sending capacities and for traffic that would be served via direct connectivity (line 8). Focusing on the largest demands, the algorithm iteratively reduces the imbalance in the remaining demand. The found demands are allocated in non-decreasing order of their size, accounting for the receiving capacity of the intermediate node. Specifically, if a demand is smaller than the remaining receiving slot capacity of the intermediate node, it is fully allocated. If the demand exceeds the remaining capacity, the capacity is equally shared among all candidate demands. Note that the algorithm does not evaluate the capacity of the slot from the intermediate node to the actual destination but relies on D3’s offloading to compensate for overloads. LLB uses a greedy approach to reduce complexity. However, to our knowledge, an optimal solution to the scheduling problem addressed is still open.

### 3.4. Data and Control Plane Components

D3 uses label-based source routing and priority queuing to forward flows to the different topology classes and to put the decisions of LLB into action. Figure 4 visualizes the components involved on the hosts and the ToRs. The example in the figure considers a configuration with one ROTOR (R0), one DEMAND-AWARE (D0), and two STATIC ports (S0, S1) on the ToR switch. All three flow classes share the uplink from host to ToR. In order to reduce interference, D3 uses priority queues on both hosts and ToR switches. STATIC flows are given the highest priority, followed by ROTOR and DEMAND-AWARE. The rationale behind this is that latency-sensitive or ‘small’ flows are sent via STATIC to avoid long waiting times for the right matching (similar to [44], which routes small flows on temporary expander graphs). For the same reason, they are assigned to the queue with the highest priority. ROTOR flows are given higher priority than DEMAND-AWARE to ensure that they are not blocked by (large) DEMAND-AWARE flows on the host uplink. Owing to their size, DEMAND-AWARE flows are considered least latency-sensitive and are therefore given the lowest priority. Queues for STATIC and DEMAND-AWARE traffic are not needed on ROTOR uplink ports, but the queue for ROTOR traffic is needed on both STATIC and DEMAND-AWARE ports to enable *offload* from the

ROTOR topology to the de Bruijn backbone. In addition, there are four more entities involved:

- The *Local* and *Non-local* buffers store packets for transmission via the Rotor part. Similar to RotorNet and Opera, each of them features a dedicated virtual queue per destination [46, 44].
- **Host agent** runs on every host and sends the individual sizes of the *Local* and *Non-local* buffers to a rack-local coordinator (**ToR agent**). It waits for *pull* messages from the **ToR agent** containing the volume per destination to be sent. When sending a packet, the host adds a *slot label* which either indicates the active Rotor matching (to send via the Rotor topology) or is 0 if the packet is to be sent via STATIC or DEMAND-AWARE ports. (Every Rotor port cycles through  $n - 1$  configurations. The slot label indicates the active configuration for all hosts.)
- **ToR agent** coordinates the Rotor packet transmission of all hosts in a rack. It can run on the ToR or on one of the hosts. It receives demand information from the **Host agents** and runs LLB. The outcome is sent to the hosts along with the slot label to be used (*pull* messages).
- FIB is a single forwarding table on the ToR for all three sub-topologies. It matches the destination IP address and the slot label to obtain the egress port. The entries for the Rotor links can be pre-computed and do not change over time (unless the number of Rotor ports changes). The forwarding entries for DEMAND-AWARE and STATIC (slot label = 0) are updated when local links change.

**Packet Forwarding:** Figure 4 illustrates the major steps taken when forwarding packets in D3. Packets from applications that should be sent via the Rotor (Green), are put in the corresponding *Local* destination queue ①. If packets are offloaded from Rotor to the STATIC and DEMAND-AWARE sub-topologies, the **Host agent** takes packets from the queue and directly sends them ②. In this case, the 0 slot label is added to the packet. Otherwise, the packets are sent according to the calculated schedule (cf. Section 3.3) with the slot label as provided by the **ToR agent**. Packets that are sent indirectly via the Rotor part are encapsulated with the address of the intermediate host. On the ToR, the packet is matched in the FIB ③ and forwarded via the medium priority queue ④.

Packets from applications to be sent via STATIC or DEMAND-AWARE (Non-Rotor), are tagged with 0 as slot label and then sent out to the ToR ①. Here, they are again matched in the FIB ② and forwarded accordingly ③. Traffic received on the ToR’s uplinks is forwarded to the destination host ① ②. When a host receives indirect Rotor traffic, it adds the packets to the corresponding *non-local* queue ③. Packets received at the final destination (host) are forwarded to the application.

### 3.5. Other System Aspects

D3’s architecture with three sub-topologies comes with several challenges regarding integration and synchronization, which we discuss in the following:

**Flow classification:** D3 requires a mechanism to classify and assign flows to one of the three sub-topologies. An option is to use flow size information as in OPERA [44], CERBERUS [24], and DUO [74], e.g., obtained via flow aging, estimation or application layer information. D3 applies this method to distinguish STATIC and DEMAND-AWARE traffic. However, the ROTOR sub-topology is particularly suited for uniform traffic patterns, like flows that belong to the shuffle phase of a map-reduce job. Therefore, D3 also uses application-level information for traffic classification. Such information may either be provided by a shim layer between the application and the network stack on the host, inserting tags that identify ROTOR traffic, or alternatively, transport layer ports are used to identify applications with uniform communication patterns.

**Transport protocols:** To leverage the capacity of the matched sub-topology, D3 uses different transport protocols for the three traffic classes: Latency-sensitive (small) flows run via the STATIC topology with NDP [29] which has shown good performance for this traffic. Flows that are transmitted via the ROTOR part are sent according to the schedules determined by LocalLB (LLB, Section 3.3). Throughput-sensitive flows use standard TCP for transmission on the STATIC and DEMAND-AWARE ports to efficiently share resources.

**Rotor Synchronization:** D3 requires two aspects of synchronization for the ROTOR sub-topology. First, ToRs must synchronize their configuration state (slot) globally across the network. This can be achieved, for example, using a global (broadcast) clock signal. An alternative solution that leverages the periodic all-to-all connectivity between ToRs (via the ROTOR-part) to synchronize the clock of a chosen primary ToR to all other ToRs, has been provided in [7, Sec. 4.4]. There, it has been shown to achieve time synchronization errors in the order of picoseconds, which is sufficiently low for the slot sizes considered.

Second type of synchronization is needed between hosts and ToRs to put into effect the LLB decisions. D3 considers a similar approach as RotorNet and Opera. Packets are primarily buffered on the hosts. Demand information is pushed *once* per slot from the `Host agent` to the `ToR agent`, e.g., with RDMA messages [46]. After having calculated the number of packets to send with LLB, the `ToR agent` pulls traffic from the host, i.e., notifies the `Host agent` about how much to send (e.g., again using RDMA messages). Other approaches may also be possible, e.g., [7]. In a recent paper, Halo [79], the authors proposed a scalable and decentralized control strategy for AWGR-type OCS. This control strategy may offer a promising mechanism for supporting and strengthening our architecture in future extensions of D3.

**Rotor Scalability:** Recent work has demonstrated that ROTOR sub-topology can be made more scalable. A recent paper by the original RotorNet authors shows that the ROTOR switch can be implemented with very low latency and high radix [45]. While this setup is using a dedicated purpose-built switch, not compatible with using passive grating as we propose, this work also contains different implementations that would assist with synchronization, and other aspects of the ROTOR network. In another line of work, the authors of Shale [4] showed that it is possible to reduce the latency of ‘round-robin’ RDCNs, such as our ROTOR-network,



by introducing a more hierarchical topology, consisting of several smaller complete graphs connected in a round-robin manner. However, this comes at the cost of throughput. D3 could be adopted to use similar topologies on larger scales.

**Forwarding tables Scalability:** When networks become larger, forwarding tables could become increasingly more complex at a rate above linear; however, for D3, we can show this would not be the case. Since D3 is composed of three types of systems, let us consider each separately. The static, de Bruijn, network employs greedy routing (as discussed in Section 3.2) and requires small forwarding tables [74]. Traffic sent to demand-aware ports is either sent at one hop using an on-demand link or using the de Bruijn ports, using greedy routing. For traffic sent on rotor ports, we only require local information for Valiant load balancing and have no issue with scalability. Thus, we assume that forwarding tables used in a possible larger future implementation of D3 are scalable.

**Dynamic Port Partitioning:** The following discusses the low-level system design aspects of D3’s dynamic port partitioning. D3 can dynamically assign ports to different schedulers to adjust the proportions of the sub-topologies. Like a “normal” reconfiguration of a DEMAND-AWARE link, reconfiguration involves changes in the FIB, which can be computed locally by the **ToR agent** or even prepared offline. In addition, the ROTOR scheduler has to reload the new schedule for the sending lasers, which can also be pre-calculated offline. The reconfigurations must be coordinated globally, e.g., with a centralized controller. All ToRs must change at the same time and, to minimize the impact on the ROTOR packet scheduling, the reconfigurations happen at the end of a ROTOR slot. D3 considers two possible types of port-to-scheduler reassignments: (1) DEMAND-AWARE to ROTOR and (2) ROTOR to DEMAND-AWARE. In (1), the **ToR agent** first removes the rules with the respective port from the FIB of the ToR and clears the queues belonging to that port (similar to a DEMAND-AWARE link reconfiguration). Then, it removes the port from the DEMAND-AWARE scheduler and assigns it to the ROTOR one by loading the new ROTOR matchings, updating the sending laser schedule and the entries in the FIB. Finally, it presents the new matchings to the packet scheduler (LLB). No updates to the host agent are required since all necessary information for it to operate is included in the pull messages. Changing a port from ROTOR to DEMAND-AWARE happens analogously in reverse order. We discuss high-level implementation of an algorithm governing dynamic port partitioning in Section 4.6.1.

### 3.6. Practicality and Cost of D3

We claim that the principles of D3 can be implemented and prototyped using the architecture of Sirius. Since the Sirius prototype (and simulation code) are not publicly available, we base our observations on a careful study of the paper [7] and personal communication with some of the authors. Recently, and in parallel to our work, the authors of ‘NegotiaToR’ [39] made a similar observation and used different link schedulers for a Sirius-based topology. Sirius is a proof of concept for a wavelength, laser-based reconfigurable topology, D3 can operate on top of other solutions that use the same concepts. Since D3 uses the same hardware as

Sirius, the main difference between the two systems is software. This includes several main components that we have already discussed: link scheduling (topology reconfiguration) via tunable lasers on both the sender and receiver, IP packet forwarding and multi-hop routing, and time synchronization. The control plane can be implemented using *local* rack-based agents and a *global* SDN-based controller that uses the static topology of D3 for control messages. As we mentioned earlier, a control plane for the demand-oblivious links (ports) already exists in Sirius, and a control plane for demand-aware links (ports) was already implemented, e.g., in ProjecToR [21]. However, unlike in Sirius, the queues at the host and ToR need to be divided into three parts, which may be achieved virtually. Lastly, a future implementation of D3, which includes dynamic port partition, would require only minor data collection and some coordination between the different ToRs and an SDN controller.

Comparing the complexity of host-side software modifications, we observe that the required effort is similar in scope: Shale [4] and Sirius [7] also utilize FPGA-based NICs, implementing custom queuing logics. For instance, to separate between direct and spraying forwarding in Shale. Also for time synchronization there are similar complexities in these work. However, we acknowledge that they operate without interaction regardless of the transport protocol. Negotiator [39] shifts complexity into the network by maintaining per-destination queues at the ToR and identifying mice flows using multi-level feedback queues. This requires ToR-level flow inspection and per-destination state in the fabric. Our approach instead keeps destination-specific state at the sender and avoids per-flow state in the network. Lastly, Opera [44] requires per-destination priority queues and advance knowledge (or estimates) of flow sizes to decide on the transport protocol, an approach very similar to ours.

Moreover, we claim that since D3 uses the same hardware as Sirius, the cost analyses of Sirius still hold. In [7], it was shown that Sirius is cost-effective compared to a classical EPS-based network (e.g., Fat-trees), i.e., Sirius can reach the same throughput at a lower cost. In turn, this claim also applies to D3, as we demonstrate next, since D3 enhances the throughput of Sirius-like systems, specifically demand-oblivious, rotor-based RDCNs. More specifically, the authors of Sirius claim that their system costs 53% of an equivalent oversubscribed EPS-based Clos. They further note that Sirius only consumes 25% of the power of an equivalent EPS based network. We note that the higher cost of EPS based networks compared to OCS-based networks is also known from other implementations. In [60], the authors report that their system requires only 70% of the initial capital cost and consumes 59% of the power of a comparable Clos baseline. With similar results in other papers [55, 65]. In Meta’s TopoOpt [70], the authors describe using reconfigurable OCS (not AWGN) to achieve a 3.5 times faster training time compared to a similarly cost fat-tree. We therefore expect this trend to hold in future implementations of D3; however, an exact estimation is left for future work.

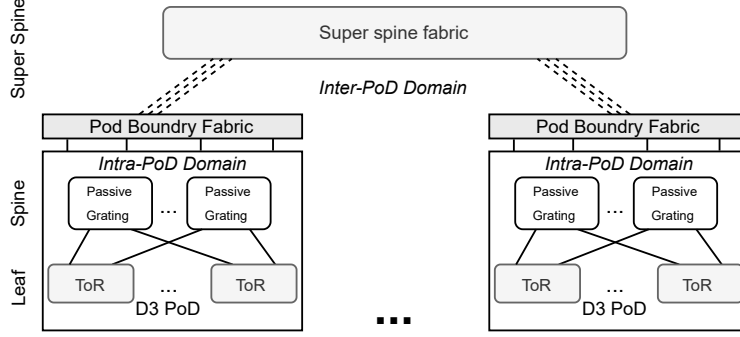


Figure 5: We envision scaling the D3 architecture such that each D3 instance forms a PoD, with multiple PoDs interconnected through a super-spine.

### 3.7. Comparison to Clos networks

Current DCN are mostly based on static Clos networks using electrical packet switches. These offer advantages such as simpler scalability, and they have access to a mature set of protocols and algorithms (congestion control etc). The static nature of the networks is also well-suited for delay-sensitive traffic, which is why we incorporate a static component in D3. However, optical circuit switches have many advantages over traditional electrical packet switches, which have been explored in several works, such as [39, 44, 24, 60, 7, 67] and many others. In particular, our offer is to use an AWGR-based switch as in Sirius [7]. These types of OCS have lower power consumption compared to EPS, as they do not require multiple optical-electrical-optical conversions, unlike EPS-based networks, which can substantially reduce power consumption. AWGR are passive and thus require no extra power at the spine layer. They are bandwidth-agnostic and thus do not require an upgrade when changing the core networks [7, 64, 56, 60, 65, 73, 79, 72]. OCS, in general, has enabled network designers to utilize larger radix switches compared to EPS [56], resulting in flatter topologies. While reconfigurability is not a necessary feature of all OCS, this innovation enables the network to achieve better connectivity with fewer switches, albeit at the cost of a more complex software and control plane, which the networking community is currently developing. Sirius, and other works such as [39, 44, 24] have compared dynamic OCS-based RDCNs networks with static ones (including Clos), showing clear benefits. We believe that further benefits will be derived as research progresses and the technology matures, for example, by developing better congestion control [37, 16].

### 3.8. Extending D3 to Hyper-Scale Datacenters

OCSs support flat leaf-spine designs at medium datacenter scale [56, 64]. However, scaling the spine to connect hundreds of racks or more still poses a challenge for a leaf-spine architectures. A typical solution to datacenter scaling is to add another layer of connectivity above the spine. This architecture, illustrated schematically in Figure 5. Here, each PoD is a leaf-spine network based on the D3 design as seen in Figure

1(a). To scale the datacenter, the PoDs are interconnected via a *super-spine*. To realize this interconnection, we assume that each ToR switch contains a set of ports for intra-PoD traffic and a different set of ports for inter-PoD communication. The ToRs are connected to a border fabric, which in turn connects to the super-spine. This concept is similar in spirit to the one taken by Meta’s MixNet architecture in [40]. An example of hyper-scale datacenters composed of several leaf-spine PoDs can be found in a recent Intel white paper [55]. Intel reports that PoDs can be added or removed from the super-spine, and they form the main building block of the hyper-scale datacenter network. We also note that Nvidia reports on leaf-spine PoDs, which can be used independently or as part of a larger network [50]. A similar architecture is used by Meta for AI training tasks [40]. However, for our case, we leave details of the exact hardware cost analysis for future work. Finally, we would also briefly mention a different approach to scaling datacenters with OCS taken by Google [42, 52, 65]; here, the OCS layer connects PoDs, while intra-PoD traffic is established using EPSs. This allows OCS to serve as the backbone of hyper-scale data centers, as each PoD can contain thousands of hosts, while the number of PoDs that the OCS layer connects is in the order of tens [63]. In principle, we would view D3 as operating at the OCS layer, connecting the different PoDs as it would connect ToRs.

#### 4. Empirical Evaluation of D3

We evaluate D3 with packet-level simulations using *htsim* [29], which has also been used in [44] and [74]. We compare D3 with two state-of-the-art RDCNs, a Sirius-like, demand-oblivious system, OPERA [44], and a demand-aware system, DUO [74], across a range of traffic patterns that are based on available empirical flow size distributions and synthetic patterns. The evaluation focuses on throughput as the main performance metric, but also investigates flow completion times and the efficiency of resource usage.

##### 4.1. Settings

We consider topologies with  $n = 64$  ToRs. Each ToR has 16 bidirectional ports, which are split equally into up- and downlinks ( $k = 8$ ) of 10 Gbps capacity each. This results in a total uplink capacity of 5.12 Tbps, which is comparable to [44, 7, 74]. Unless stated otherwise, the physical reconfiguration delay is 100 ns for ROTOR switches and 1 ms for DEMAND-AWARE switches, with a  $1.7\mu s$  guard period on the ROTOR part to empty the links before reconfiguration. Table B.4 (in Appendix) summarizes the used reconfiguration periods. The reconfiguration periods are chosen such that the duty cycles in both dynamic topology parts are similar to previous work [44, 74]. We compare the following three systems:

**D3:** uses the hybrid topology as presented in Section 3. Throughout the evaluation, we vary the proportions of the parts denoted by the tuple  $(k_s, k_r, k_d)$ . Scheduling of DEMAND-AWARE links follows the approach for Duo (see below). Similarly to previous work [44, 74], we assume full knowledge of the flows’ sizes at any point in time. The allocation of flows to scheduling classes is based on two criteria. First, we assume

application-level information to identify (uniform) traffic for the ROTOR part using RLB (see also the traffic description later). Second, a size threshold of 1 MB separates traffic for only the STATIC part using NDP and traffic for the DEMAND-AWARE part using TCP. Lastly, the offloading parameter is  $d_{\text{off}} = 1$  packet, which is the volume that can be sent in a single slot and a single path, assuming all-to-all traffic and fair-share.

**Duo [74]:** has been presented in previous work, but can be seen as a specific configuration of D3 with  $k_r = 0$  (no ROTOR part). We consider a configuration with  $k_s = 2$  and  $k_d = 6$ . The DEMAND-AWARE links are greedily scheduled based on the remaining demand volume between the ToR pairs according to Algorithm 2 in [74] with a threshold of 10 MB. The algorithm has full knowledge of the flows’ sizes upon their arrival. The routing uses the properties of de Bruijn-based greedy routing, i.e., paths combining STATIC and DEMAND-AWARE links are possible. Similar to D3, flows  $< 1$  MB use NDP and larger flows use TCP. Queues in the two systems above can hold 50 data packets of size 1500 B. We note that in the setting of our paper, a pure demand-aware configuration, i.e.,  $(0, 0, 8)$ , would fail to deliver good flow completion times. Since small flows cannot amortize the reconfiguration time of demand-aware circuits, removing the static ports will greatly increase the FCT for small flows, a critical measure for this class of flows. Thus, in our setting, Duo represents the maximal allocation of demand-aware ports. **Opera/Sirius [44]:** is a demand-oblivious, dynamic topology and serves as the second baseline. Since an implementation of Sirius is not available, we use Opera as the closest representative of a Sirius-like system. It periodically cycles through a specifically generated set of matchings that maintains an expander graph at every time instance. OPERA also divides the flows into low-latency (i.e. small flows) and bulk traffic. Low-latency traffic is forwarded via the temporarily static expander part of the topology with NDP, whereas bulk traffic is scheduled and sent with the RotorLB protocol and scheduling [44, 46]. The evaluation uses their default configuration for queue sizes  $(8 \cdot 1500 \text{ B})$  and the threshold for bulk traffic is 15 MB.

**Traffic:** We consider an online traffic scenario where flows arrive over time according to a Poisson process. To demonstrate that D3 is particularly suited for traffic patterns that mix skewed demands with uniform patterns, we create traces that contain traffic from two distributions and vary the shares of the patterns. We denote by  $x$  the *share* of the skewed traffic. For the skewed part, connection pairs are sampled uniformly at random. The flow sizes are sampled from available empirical flow-size distributions obtained from traffic observations in DCNs. We focus on DATAMINING [22] in this paper but yield similar results for two other distributions [54, 3]. The DATAMINING distribution is commonly used in related works to represent realistic traffic load distribution [4, 44, 37, 24, 74] and others. The uniform pattern is created by a matrix with one flow of size 112.5 KB for each host pair in different racks. The flows of a matrix arrive over time, with load  $L$  controlling the arrival rate. We report on 10 runs per setting.

**Metrics of Interest:** D3 targets at throughput as the main metric of interest. Since its design covers the stack from topology, routing, and parts of the transport layer, we consider the throughput on the transport

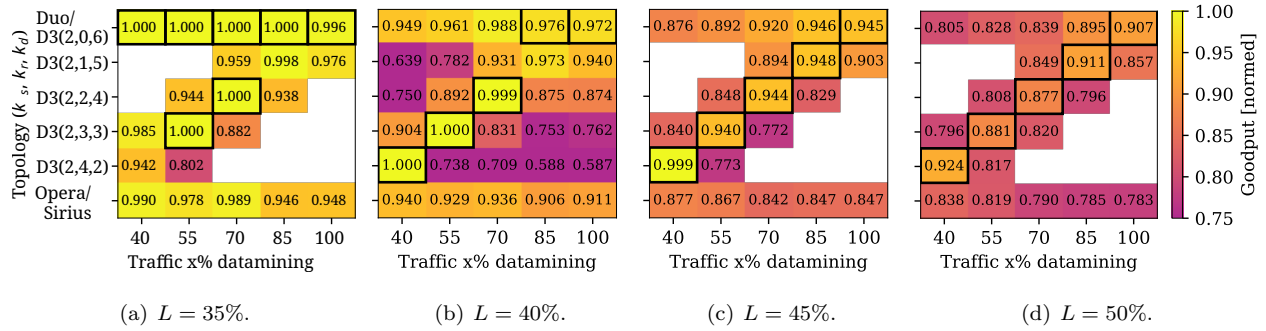


Figure 6: The average goodput over 1s of simulation. The heatmap compares topology configurations (Y-axis) and traffic shares (X-axis) for different loads  $L$ . Values are normalized to the offered traffic. The best result is highlighted with a thick border. The white regions are excluded for clarity, they are far from optimal.

layer, i.e., the goodput. This takes into account transport layer behavior and overhead (e.g. retransmissions from reconfigurations). Note that, in this section, we may use the terms “goodput” and “throughput” interchangeably. Furthermore, we investigate the impact on individual flows’ completion time (FCT), which is particularly relevant for latency-sensitive flows.

#### 4.2. Throughput Evaluation

We start with evaluating the throughput of D3.

##### 4.2.1. Traffic Loads and Traffic Mixes

Figure 6 visualizes the average goodput of 1 s of simulation for different topology configurations and traffic mixes. The values are normalized to the offered load. A value = 1 means that the topology can fully serve the offered traffic. The skewed traffic is sampled from DATAMINING. Comparing the different loads (sub-figures), we note that the normalized values decrease slightly with increasing load. For  $L = 35\%$  (Figure 6(a)), the number of cells = 1 is the highest, whereas for  $L = 50\%$  (Figure 6(d)), none of the configurations can fully sustain the offered traffic. This is expected as the congestion in the topologies increases.

Looking at the individual heatmaps (e.g., Figure 6(b)), the performance of the topology configurations varies with the traffic share between DATAMINING and uniform. That is, for each share, there is one best configuration. Moreover, topology configurations with a small number of ROTOR links serve better traffic mixes with higher  $x$ . This aligns with the conclusions made in prior work [24]. The share of ROTOR links approximately corresponds to the share of uniform-size flows in the traffic. For instance, the configuration (2, 2, 4) dedicates 75% of the resources to STATIC and DEMAND-AWARE and performs best for  $x = 70\%$  in which 70% of the traffic uses these sub-topologies. Over-provisioning the ROTOR part reduces the goodput more than over-provisioning the DEMAND-AWARE one. Compared to D3, OPERA performs worse for all traffic mixes (and load levels) considered. For instance, for  $L = 40\%$ , OPERA achieves approximately 6%

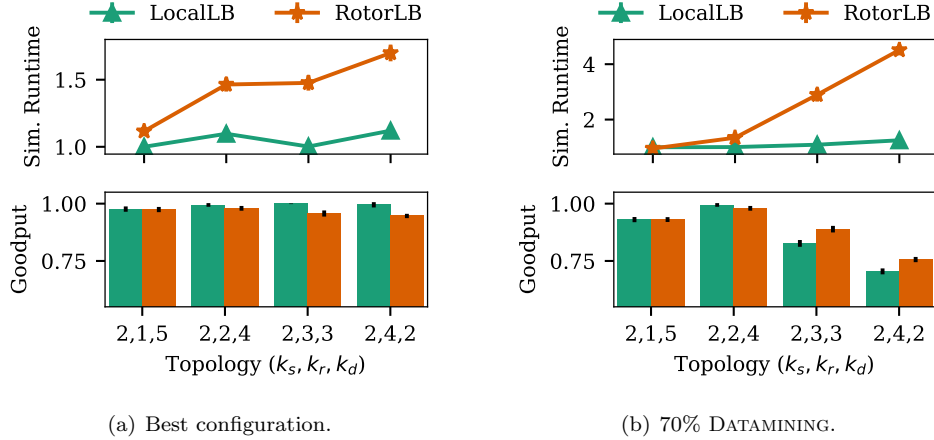


Figure 7: Comparison of D3's scheduling (LocalLB) and RotorLB scheduling for load  $L = 40\%$ .

lower goodput than D3. DUO, which resembles a special case of D3 (with  $(2,0,6)$ ), performs the best for the high  $x$ . Overall, choosing the correct configuration for D3 is important to maximize its performance. If not stated otherwise, the following analyses use  $L = 40\%$  and  $x = 70\%$  sampled from DATAMINING.

#### 4.2.2. LocalLB Throughput & Complexity

In the previous analyses, D3 uses the LocalLB proposed in Section 3.3. This scheduling can run rack-local, which reduces the synchronization overhead compared to running D3 with the RotorLB scheduling [46, Algorithm 1]. Figure 7 shows the goodput and simulation runtime (wall clock time) of both scheduling approaches within D3 for  $L = 40\%$ . We acknowledge that the simulation runtime does not reflect the actual synchronization effort, which also depends on hardware characteristics, but can give a first indication. The figure compares two cases: (a) with the topology matching the traffic share, i.e.,  $(2,1,5)$  is run with  $x = 85\%$ ,  $(2,2,4)$  with  $x = 70\%$  etc., and (b) with  $x = 70\%$ . For all configurations, the differences in goodput between D3 and RotorLB are  $< 10\%$ . For the matching traffic, we observe that LocalLB achieves slightly higher goodput than RotorLB and for the fixed share, the opposite is the case. LocalLB consistently has lower run times than RotorLB. The difference increases with the number of ROTOR ports and emphasizes the gain of avoiding inter-ToR synchronization for indirect traffic.

#### 4.3. Sensitivity Analysis

D3's advantages also persist in scenarios with different system parameters and traffic characteristics. The following part focuses on the impact of the reconfiguration times of the DEMAND-AWARE topology and also shows that D3 is fairly robust to delayed or erroneous flow size information. D3 outperforms the reference solutions also with higher link capacities, varying offloading from ROTOR to the DEMAND-AWARE topology, other loads, other empirical flow size distributions (HADOOP [54] and WEBSEARCH [3]) and less bursty traffic.

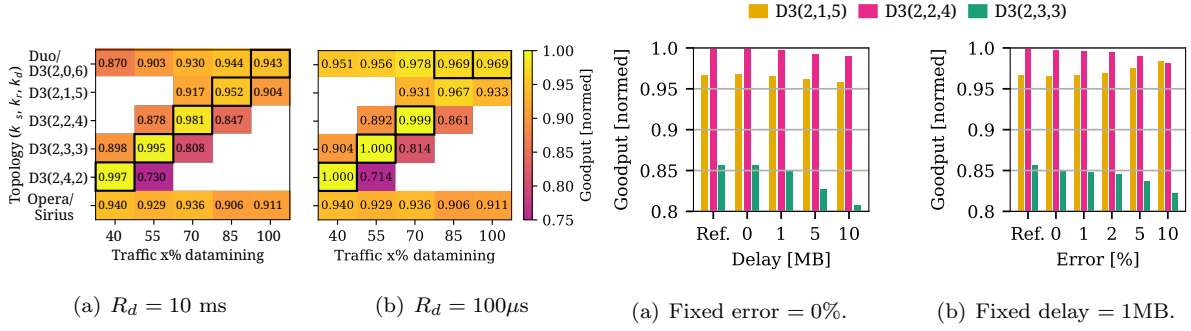


Figure 8: Goodput averaged over 1s of simulation. Comparison of reconfiguration times of the DEMAND-AWARE links with fixed duty cycle. The heatmap compares topology configurations and traffic mixes.

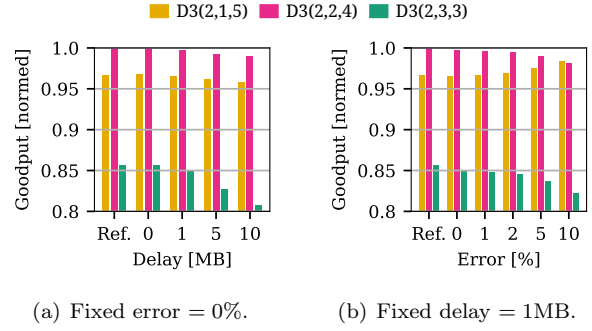


Figure 9: Mean goodput for scenarios with restricted flow-size information. (a) varies the delay of the information and (b) varies the error.

#### 4.3.1. Reconfiguration Times

Figure 8 illustrates heatmaps similar to those in Figure 6 for  $R_d = 10$  ms (Figure 8(a)) and  $R_d = 100\mu s$  (Figure 8(b)). The reconfiguration period is adjusted so that the duty cycle remains constant. The load is  $L = 40\%$ . On a macroscopic level, we observe a similar behavior as before: matching the topology configuration to the traffic mix maximizes the achieved goodput. When comparing the values in detail, no consistent impact of the reduction of  $R_d$  (and the implied reduction of the reconfiguration period) is observable. Some combinations have higher and some have lower achieved goodput. Note that the values for OPERA are not affected when changing  $R_d$ .

#### 4.3.2. Classification Errors and Delayed Information

D3 requires information about the flowsize readily available to classify flows and up to here, the evaluation assumed perfect knowledge. However, such information might be imperfect or delayed. Therefore, and in order to demonstrate D3's robustness, we relax this assumption along both dimensions: 1) delaying information about the total flowsize, 2) introducing errors to the flow classification.

For the first case, Figure 9(a), we delay the flowsize information in the simulation so that the first part of the flow is handled as a small flow. The full information flowsize is then available after the time it would take to transmit the first part over an ideal link. This can also be interpreted as some kind of flow aging. When the full flowsize information is released (emulated as a second flow arrival), the flow can be properly classified and forwarded on the fitting sub-topology. The figure compares the delays of 1, 5, and 10MB which correspond to 667, 3334, and 6667 packets respectively. The figure shows the results without flow classification errors. Note that since the traffic generation for this analysis is not exactly the same as before, attention should be put on the relative differences and a 1:1 comparison of the numeric values to other figures is not meaningful. Comparing the results to the reference case, the goodput reduces slightly as the flow size



information delay increases. This is expected as more traffic is routed through the static topology part. However, the reduction amounts to only 2 – 3% for 10MB delay (which is already larger than most flows).

The second relaxation (Figure 9(b)), is an additional error in the flow classification (on top of a fixed flowsize information delay of 1MB). Here,  $y\%$  of the uniform flows for the Rotor-topology are mis-classified and forwarded via the static topology part. The goodput of the best topology configuration for  $x = 70\%$  (2,2,4) continuously reduces as the error increases. The reason is that the actual (allocated) traffic share changes. The share of traffic allocated to the STATIC and DEMAND-AWARE topology increases. Therefore, we also observe that (2,1,5), which dedicates more resources to those topology parts, shows an increase in goodput, up to the point that the ranking changes at 10% error. Lastly, the goodput of (2,3,3) reduces for the very same reason as described above.

#### 4.3.3. Link Capacity & Offloading

To illustrate that the benefits of D3 also persist for higher link capacities, Figure 10 shows the goodput achieved for scenarios with higher link capacity. The detailed parameters are summarized in Table B.4 (in the appendix). The load is  $L = 40\%$  with the share of DATAMINING  $x = 70\%$ . The figure shows averages of 10 runs.

Overall, we can observe a similar behavior as for 10G. The matching topology achieves the highest throughput. The performance deviations of D3 between 10G and 100G are within the range of single-digit percentages, but there is no consistent pattern. 50G shows slightly different behavior than 10G and 100G for the configurations close to the matching one. Here, the goodput reduces almost symmetrically for too few or too many ROTOR links, whereas it is asymmetrically for 10G and 100G. We leave a more detailed investigation for future work. In summary, we conclude that the gains of D3 persist also for other link capacities.

Figure 11 visualizes the impact of the offloading threshold. The link capacity is 10G. The figure compares three values 1, 2, and  $\infty$  and also shows OPERA as a reference. The values are normalized to the best configuration with  $d_{\text{off}} = 1$  (2,2,4). The achieved goodput varies with the threshold. However, the ranking among the topology configurations persists. The specific impact depends on the topology configuration. For instance, for (2,2,4),  $d_{\text{off}} = 2$  results in slightly lower goodput than  $\infty$  or 1. In contrast, for (2,1,5), the impact is negligible and for (2,3,3), we observe that the goodput decreases significantly as the threshold is reduced and more traffic is offloaded. Overall, the amount of offloading should be optimized.

#### 4.3.4. Performance for Other Traffic Distributions

Figure 13 summarizes the performance of D3 when evaluated with traffic generated based on the HADOOP [54] and WEBSEARCH [3] distributions. The figure shows the relative gain in goodput when using D3 compared to the indicated reference configuration. The gain is calculated as  $\frac{G_{D3} - G_{ref}}{G_{ref}}$ . Note that (2,0,6)

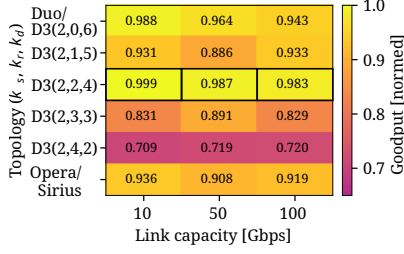


Figure 10: Heatmap of the normalized achieved goodput against the link capacity, and topology configuration.

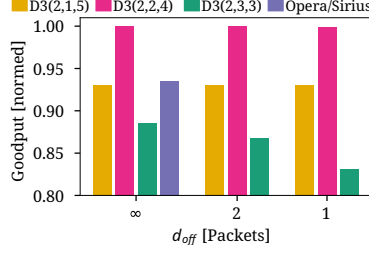


Figure 11: Bar of the normalized achieved goodput against the LocalLB offloading threshold ( $d_{off}$ ) and topology configuration.

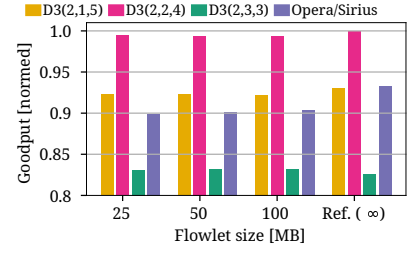


Figure 12: Barplots of the mean goodput for traffic with large flows split into chunks (flowlets).

(DUO) is also considered as part of D3 here.

The reference configurations are DUO (2,0,6) and OPERA, respectively. First, for DUO (Figure 13(a)), we observe that the gain reduces for all distributions when  $x$  increases. For higher shares, the optimal configuration has smaller  $k_r$  and becomes more similar to (2,0,6). In fact, for  $x = 100\%$ , (2,0,6) is the best configuration.

Compared to OPERA (Figure 13(b)), D3 shows improvements up to 10% when skewed traffic is sampled from the DATAMINING or HADOOP distribution. The numbers vary depending on the exact share. For WEBSEARCH, the gain strongly grows with  $x$  (the upper-end is cut. The max gain is 112% at 100% WEBSEARCH). Also, in prior work [44, 74], OPERA’s throughput was shown to collapse if the amount of traffic routed via the expander part increases, as is the case with the WEBSEARCH distribution. As a general trend, we observe that the gains decrease with the average flow size of the distributions (DATAMINING has the highest, followed by HADOOP and then WEBSEARCH). The intuition is that HADOOP and WEBSEARCH are, per se, less skewed than DATAMINING, which overall benefits OPERA. In summary, the gains from D3 persist for other skewed traffic distributions.

#### 4.3.5. Impact of Reduced Burstiness

In previous evaluations, the entire volume of the flow is available for transmission upon arrival, resulting in highly bursty traffic when large flows arrive. To assess the performance of D3 in a less bursty scenario, we split large flows into several flowlets (of a given size) arriving sequentially with a certain delay. The flowlet inter-arrival time is calculated from the ideal transmission time. The values are normalized to the best solution (2,2,4) in the reference case in which the flowlet size is  $\infty$ , i.e., the entire flow arrives at once. Figure 12 varies the size of the flowlets. When matching the topology to the traffic, almost no impact of the flowlet size is observable. Also, the ranking between the topology configurations persists. For the not-matched configurations (2,1,5) and (2,3,3), there are slight variations in the achieved goodput. Also for OPERA, the goodput decreases with the flowlet size.

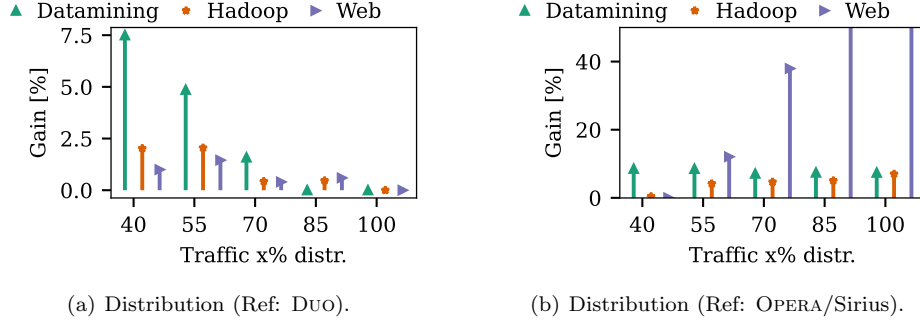


Figure 13: Comparison of gain in goodput when using D3 for different flowsize distributions for the skewed traffic. The reference topology is indicated in the caption.

#### 4.4. Individual Flows' Performance

This subsection evaluates the impact on the individual flows.

##### 4.4.1. Flow Completion Times (FCT)

Figure 16 visualizes the FCTs as a function of the flowsizes for D3 (2,2,4), DUO (2,0,6) and OPERA. The flows are separated into three groups for the sake of readability. The dashed lines indicate the optimal transmission time accounting for queues and propagation time (using the calculations provided by [44]). For small flows ( $\leq 100$  KB, Figure 16(a)), we consider the 99%-ile to better understand how this latency-sensitive traffic is handled. For all sizes, D3 achieves the smallest FCT, followed by OPERA and DUO. Recall that small flows use only the STATIC topology. Compared to D3, DUO's performance suffers from the uniform traffic that cannot be served well over the DEMAND-AWARE links and creates congestion on the STATIC links.

For medium-sized flows (99%-ile, Figure 16(b)), D3 consistently performs better than DUO for the same reason as for the small flows. D3 also outperforms OPERA until the point where D3 starts to classify flows for the DEMAND-AWARE part (1 MB). From there on, flows are using TCP, and there is a strong increase in the FCT, an effect that has already been observed in previous work, e.g., in DUO [74] and similarly also for OPERA [44] (for flows  $> 15$  MB). A special point of interest is the flows of size 112.5 KB, which arrive according to uniform matrices. All systems show an increased FCT here. OPERA is the best, followed by D3 (with the optimized configuration) and DUO (2,0,6). This steep increase can be explained by the fact that these flows do not fit in a single ROTOR slot and need multiple cycles of the ROTOR sub-topology to be transmitted.

The situation changes for FCTs of large flows ( $\geq 100$  MB, Figure 16(c)). Since throughput matters more for large flows, we consider the median value here. D3 (2,2,4) performs best followed by DUO (2,0,6) and OPERA. Specifically, the distance between D3 and DUO is smaller compared to OPERA; in fact, the FCTs for DUO and D3 overlap for most flow sizes. This relates to our previous observations regarding the achieved goodput. In summary, integrating ROTOR into a purely demand-aware topology improves the FCT

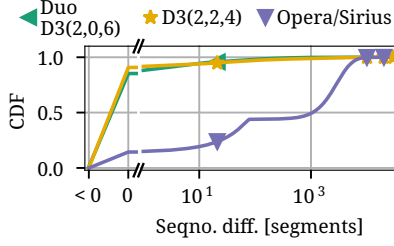


Figure 14: Difference in sequence number of received packets.  $L = 40\%$ ,  $x = 70\%$ .

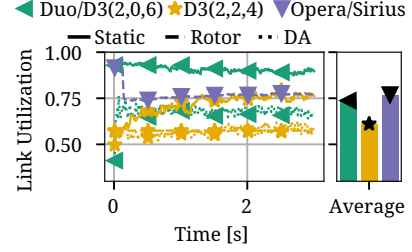


Figure 15: Link utilization over time.  $L = 40\%$  and  $x = 70\%$ .

for all flow sizes. Compared to OPERA, D3 trades off the benefits of small and large flows against those of medium-sized and ROTOR flows.

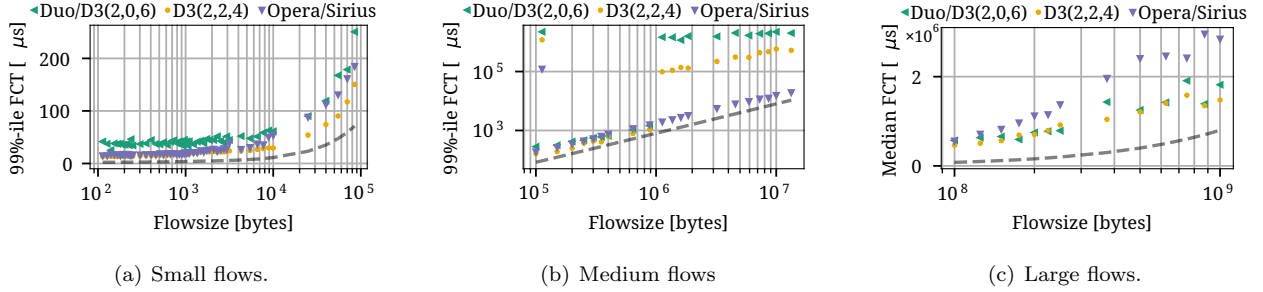


Figure 16: Comparison of FCT. Flows separated into three groups. 99%-ile for small flows (a), medium flows (b), and the median for large flows (c).  $L = 40\%$ ,  $x = 70\%$ .

#### 4.4.2. Packet Reordering

To take a different perspective on the impact on the individual flows, Figure 14 illustrates the difference between the expected and received sequence numbers of the packets at the receiver. Thereby, it hints at the packet reordering experienced in the different topologies. The abscissae are divided into values  $< 0$  (useless re-transmissions),  $= 0$ , and more fine-grained for values  $> 0$  (reception of re-ordered packets). Comparing the systems, we observe that the behavior of (2,0,6) and D3 (2,2,4) is similar. For both,  $> 80\%$  of the packets arrive in order. This is fundamentally different for OPERA for which  $< 20\%$  of the packets arrive in order. The shape of the remaining curve has a break around 79 packets, which corresponds to the number of packets needed to transmit the flows from the uniform traffic (of size 112.5 kB). We conclude that OPERA requires more effort at the receivers to forward packets in order to the applications.

#### 4.4.3. Effect of Offloading on Small Flows

Offloading second-hop traffic from ROTOR to the STATIC could theoretically impact the FCT of delay-sensitive flows. To analyze the impact, Figure 17 shows the relative difference between the 99-th percentile

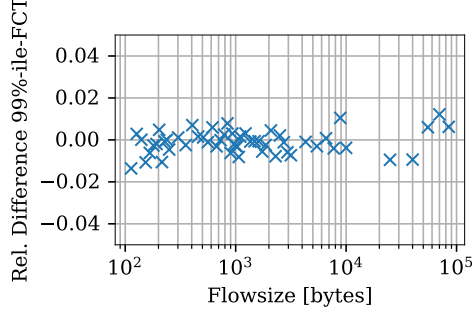


Figure 17: Relative difference of completion times of small flows for configurations with ( $d_{off} = 1$ ) and without ( $d_{off} = \infty$ ) offloading. The figure compares the 99th percentile of runs and shows averages across 10 runs.

of the completion times by flow size. The observed effect is minor, with values ranging from  $-1.4\%$  to  $1.2\%$ , and there is no clear dependency on the flow size. This limited effect can be attributable to the weighted fair queuing mechanism. Specifically, offloading migrates packets from the ROTOR to the de Bruijn topology, thereby increasing traffic volume on the De Bruijn and slightly shifting the effective transmission timing for latency-sensitive queues. With some flows experiencing increased delay and others reduced. The mean difference over all flow sizes is close to zero.

#### 4.5. Resource Utilization

Figure 15 shows the link utilization over time and on average for a single run. For each topology, the present link types are shown separately (OPERA has only ROTOR, DUO has only STATIC and DEMAND-AWARE, and D3 has all three). After the initial transient phase, the utilizations are stable over time and vary only a little. For DUO (2,0,6), the utilization of STATIC links is around 90%, whereas DEMAND-AWARE are utilized by 70%. OPERA’s ROTOR are  $\approx 80\%$  utilized. For D3 (2,2,4), ROTOR and DEMAND-AWARE have a similar utilization of 60%, and STATIC are utilized by 80%. We note that D3 has the lowest utilization on average but achieves the highest throughput, demonstrating the benefits of matching traffic and topology in the dimension of resource efficiency.

#### 4.6. Dynamic Traffic & Partitioning

As a last aspect, we assess the benefits of dynamic partitioning in D3. Figure 18(a) shows the goodput over time and the average of a simulation with varying traffic mix. Specifically,  $x$  increases from 55% for  $< 2s$  to 70% ( $2s \leq t < 4s$ ) to 85% ( $4s \leq t < 6s$ ) and finally reduces to 70% again for  $\geq 6s$ . For each of the three traffic mixes, the figure shows the goodput of the matching configuration (2,3,3), (2,2,4), and (2,1,5), respectively. For these three static baselines, we see in the period with matching traffic that they achieve the offered load (after some transient phase) but drop in goodput when not matching the traffic. For instance, (2,3,3) significantly drops after 2s. In contrast, for dynamic partitioning, the achieved goodput aligns with

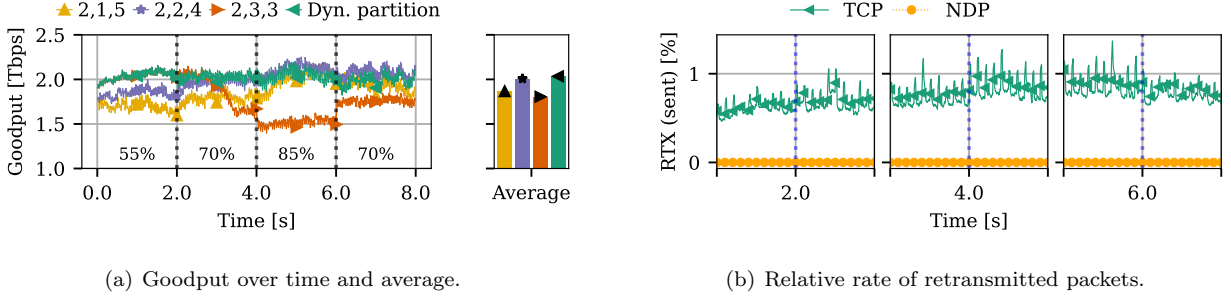


Figure 18: (a) Goodput over time and average. Dashed lines indicate changes in the traffic mix. (b) Retransmission rate per transport protocol at the sender relative to the successfully received volume per 1ms window over time. The figure zooms into the changes in the traffic mix at 2, 4, and 6 seconds (dashed lines).

the offered load throughout the whole run. This becomes also evident from the average values. Note that the shown example is hard-coded and only demonstrates the benefits of applying dynamic partitioning.

Figure 18(b) illustrates the rate of retransmissions per transport protocol (TCP or NDP) captured at the senders normalized by the rate of total transmitted data. By design, there are no retransmissions for RLB. The abscissa of the figure is divided to zoom-in to the time ranges close to a change in the traffic pattern, showing  $\pm 0.5$ s around the change. All topology configurations show retransmission rates in a similar range. For NDP, there are no retransmissions. Moreover, in all cases, we observe periodic spikes due to reconfigurations of the DEMAND-AWARE links, when packets in the affected queues are dropped. The reconfiguration of the sub-topologies' sizes (at  $t = 2, 4, 6$ s) uses the same approach when re-assigning the link scheduler, hence, we observe the same effects.

#### 4.6.1. Dynamic Port Partitioning Algorithm

As discussed, we have not implemented a port partitioning algorithm for the different port types (STATIC, ROTOR, DEMAND-AWARE). In this section, we outline how our results and insights from related work inform a design philosophy for such an algorithm. At its core, dynamic port partitioning requires an online approach. Before addressing the online setting, we first describe an offline version that provides what we believe to be a good approximation of the optimal port partitioning.

**Offline port partitioning:** Our results in Figures 6, 8, and 18(a) suggest a simple rule: the best port partitioning is the one that most closely matches the *distribution of load* across two variable port types. Recall that our evaluation considered two distinct traffic classes: DATAMINING traffic, which primarily uses DEMAND-AWARE ports, and uniform traffic, which primarily uses ROTOR ports. While the number of STATIC ports remains fixed ( $k_s = 2$ ), the optimal fraction of ROTOR ports matches closely with the fraction of uniform flows in the workload and similarly with DEMAND-AWARE ports and the share of DATAMINING traffic. For example, a 100% DATAMINING workload performs best under the DUO configuration (2, 0, 6). A workload

comprising 85% DATAMINING traffic is best served by the (2, 1, 5) configuration, in which approximately 83% of non-STATIC ports are DEMAND-AWARE. Likewise, a 50% DATAMINING traffic mix performs best under the (2, 3, 3) partitioning. Across our experiments, deviations from this pattern are rare and minor. Although we do not claim to have identified the rule that yields a provably optimal port partitioning, the empirical relationship above appears to be a strong approximation of it, while the optimal rule remains an open problem. For now, the goal of the dynamic port partitioning mechanism we envision for future work is, therefore, to approximate this rule as closely as possible in order to maximize goodput.

**Dynamic online port partitioning:** In the online setting, we assume that flow classification to each port type has already been determined by an independent algorithm; this assumption is similar to the one made in this paper, as described in Section 3.5. Our dynamic port partitioning algorithm aims to adjust the share of each port type, STATIC, ROTOR, and DEMAND-AWARE, so that it reflects the observed traffic proportions of small, uniform, and large flows, respectively, while minimizing the frequency of changes. We denote our hypothetical algorithm for *Dynamic Online Port Partitioning* as DOPP. The algorithm would be based on a centralized SDN controller, which would also serve other purposes. The SDN controller may send control messages using the static topology. DOPP receives information on the current flow load distribution and current port share and outputs a decision on how and whether to update the port partitioning. The current flow distribution can be gathered using the SDN controller. Assuming that flow information is encapsulated in the packet header, the controller would only need to sample a relatively small number of packets randomly to gather a good approximation of the load share (law of large numbers). Traffic information is commonly collected in datacenter networks via sampling; we therefore believe the overhead for this would be small or negligible.

After receiving the flow distribution, DOPP will assign the total port budget  $k$  among three groups:  $k_s$ ,  $k_d$ , and  $k_r$ . We note that, since the static network requires a certain number of switches to function, the algorithm will always allocate at least two STATIC ports. DOPP can decide to redistribute ports at any time. However, this incurs two types of costs. The first is that packets may be dropped, and parts of the network may temporarily become unavailable. While the best frequency to perform a re-partitioning is unknown, Figures 18(a), 18(b) and 17 suggest that this cost is small, and likely can be amortized even at time scales on the order of one second and longer. The second cost is the cost of choosing a wrong partition and serving traffic sub-optimally. Since, other than in some cases<sup>4</sup>, future traffic is unknown, DOPP might select a bad partition based on past statistics, which it will ‘regret’ since it has caused a performance degradation. In the context of online learning and algorithms, this performance degradation is formalized as *regret* [59], which quantifies how much worse the algorithm performs compared to the best

---

<sup>4</sup>Some ML training algorithms, such as a mixture of experts, have an all-to-all communication phase[30, 40]. Since this is predictable, DOPP could change all ports to ROTOR ports before this phase to achieve best performance.

fixed decision it could have made in hindsight. For example, in 18(a) we see an optimal algorithm denoted as *Dyn. partition*, which changes port partitioning as the traffic share changes, while the other are less than ideal. The regret each algorithm suffers is the distance in goodput between each algorithm, as described in the bar chart on the right. Conveniently, we also observe that this regret is proportional to the degree of difference between the correct partitions, as evident in our results. We are currently unaware of any known online regret minimization problem and algorithm that match our case perfectly; however, we speculate it can be reduced to a convex hull optimization problem [64] ( Since all possible port partitions form a square in  $\mathbb{R}^2$ ).

To summarize, our work on D3 allowed us to gain a solid understanding of the needs of the online algorithm. We believe that such an algorithm would be feasible, scalable, and not computationally challenging. However, extensive work would be needed to establish DOPP as a competitive algorithm, which we reserve for future work.

## 5. Theoretical Modeling

This section provides an analytical explanation for the superiority of D3 over OPERA/SIRIUS and DUO in terms of *throughput*. In this section, we will analyze throughput indirectly by studying the *demand completion times* (DCT) of these three systems in a simplified model that captures the essence of the systems. In [24], a direct connection between the DCT, the time to send all the traffic of a demand matrix, and the throughput was proved; briefly, it was shown that, under some assumptions consistent with ours, there exists an inverse relationship between DCT and throughput of a network for a given demand matrix. Thus, DCT can be directly mapped to throughput, and a shorter DCT implies a proportionally higher throughput. This led to a novel method that allows us to account for reconfiguration times when analyzing the throughput of dynamic networks.

To compare the above three systems, we consider the following cases: i) We emulated DUO using what we call *da-net* where all traffic is sent via the DEMAND-AWARE link scheduler (matchings) with a reconfiguration delay of  $R_d$ . ii) We emulate OPERA/SIRIUS using what we call *rotor-net* where all traffic is sent via the ROTOR link scheduler that rotates between  $n - 1$  predefined (oblivious) matchings with a reconfiguration delay of  $R_r$ . iii) We emulate D3 using what we call *mix-net*, a simplified D3-based system where traffic can be split (in time) between the DEMAND-AWARE link scheduler and the ROTOR link scheduler. We make some simplifying assumptions when comparing the systems.

In our analysis, we only consider flows that are *not* time-sensitive. We assume that all time-sensitive flows are handled in all three systems via an identical STATIC topology that uses multi-hop routing. Therefore, the DCT of time-sensitive traffic is equal across all three systems and will not affect the results. To focus our theoretical discussion on the differences between each system, we remove explicit mentions of STATIC



topology from our analysis.

Following previous work [31, 49, 67], we model the traffic as a *saturated* demand matrix  $M$ , i.e., a scaled doubly stochastic matrix where the sum of each row and column is equal to the node’s capacity (depending on the link rates and the time frame that the matrix captures).

We assume that all systems have a *single* dynamic port in the ToRs, namely, ToRs are connected using a *single* matching, but the topology scheduling can change over time. Since the number of switches corresponds to the number of ToR ports, this aligns with the ‘single large switch’ abstraction used in related works, which demonstrates an equivalence (for the propose of analysis) between a network with a single  $n$  radix switch and a network with  $k$  switches, each of the same radix, as shown in [68, 20, 7, 77] and more recently in [72, 53]. Thus, our results are independent of the number of switches (ports) in the network. Theoretically, this follows from our assumption that the demand matrix is a scaled doubly stochastic matrix: if the number of ports increases by a factor  $x$ , the network capacity also increases by  $x$ ; this, in turn, would increase the size of the demand matrix by the same factor, preserving our result. In our analysis of the DCT, we assume that delay is solely due to transmission and reconfiguration times, and we neglect any delays resulting from packet loss or congestion. We now consider the DCT for *da-net*, *rotor-net*, and the simplified D3 in turn.

**DCT of *da-net* and BvN Decomposition:** Similar to previous work [67, 13], we model the DCT of *da-net* using the Birkhoff-von Neumann decomposition [10] and single-hop forwarding. Briefly, the Birkhoff-von Neumann (BvN) theorem guarantees that for any doubly stochastic matrix,  $M$ , there exists a matrix decomposition to a set  $P(M) = \{P_1, \dots, P_m\}$  of  $m$  different permutation matrices (each row and each column has exactly a single 1 entry and all other entries are 0), each with a corresponding non-negative coefficient  $\alpha = \{\alpha_1, \dots, \alpha_m\}$ . The sum of the coefficients satisfies the condition  $\sum_{i=1}^m \alpha_i = 1$  when the matrix is doubly stochastic or  $|M|/n$  when the matrix is saturated and  $|M|$  is the sum of all entries in the matrix  $M$ . The linear combination of these creates the original matrix that is:  $\sum_{i=1}^m \alpha_i P_i = M$ . Given that the DEMAND-AWARE link scheduler can emulate any matching, we can individually transmit each matching in  $P$ . Transmitting each  $P_i$  takes  $\frac{\alpha_i}{r}$  plus the reconfiguration time  $R_d$ . The DCT for *da-net*, a demand matrix  $M$ , an algorithm BvN that computes a decomposition  $\mathcal{P}(M)$ , and a set of coefficients  $\alpha$ , is the sum of transmission and reconfiguration times of each permutation:

$$DCT_{da}(M, \text{BvN}) = \sum_{i=1}^m \left( \frac{\alpha_i}{r} + R_d \right) = \frac{|M|}{nr} + mR_d. \quad (1)$$

While it is simple to find a decomposition of an order of  $O(n^2)$  terms, the problem of minimizing the number of terms in the decomposition is known to be NP-complete [19]. Note that a minimal number of terms will also minimize the total reconfiguration time and, therefore, the DCT.

**The DCT of *rotor-net*:** Recall that *rotor-net* uses faster reconfigurations to rotate between  $n-1$  predefined matchings, which, when combined in a full cycle, form a complete graph [46]. During each slot, there is a

single active matching on which packets are sent (recall that for now, we have a single ROTOR link scheduler or port). Each source-destination pair has to wait for all other  $n - 2$  matchings to rotate until it is available again.

In practice *rotor-net* and similar projects such as Sirius [7] are demand oblivious and use Valiant load balancing [66] routing on *all* packets. In this case, the fraction of direct single-hop traffic will always be 0 and all traffic is sent via two hops. Therefore, to approximate the expected DCT for *rotor-net* in the following sections, we assume that it uses Valiant routing, and we denote it as **Val**. That is, for a demand matrix  $M$  its expected DCT on *rotor-net* is

$$DCT_{rot}(M, \mathbf{Val}) = \frac{2}{\eta r} \frac{|M|}{n}. \quad (2)$$

where  $\eta = \frac{\delta}{\delta + R_r}$  is the *duty cycle*. This bound was previously claimed: informally [46, 7] and formally [24].

**DCT of *mix-net* and BvN Based Partition:** In the above, we provided bounds for two extreme cases: either the whole traffic,  $M$  is sent via *da-net* using only DEMAND-AWARE link scheduling, or the whole traffic is sent via *rotor-net* using only ROTOR link scheduling.

The basic idea of D3 and here *mix-net*, is that a partition of the traffic in  $M$  into two parts may lead to better DCT results, namely to split  $M$  into one sub-matrix that will be sent using DEMAND-AWARE link scheduling and a second sub-matrix that will be sent using ROTOR link scheduling.

We extend the classic BvN problem to the problem of DCT minimization on *mix-net*. As before, let  $M$  be a saturated demand matrix. We can use the BvN theorem to decompose  $M$  into a set of at most  $m \leq n^2$  scaled permutation matrices  $\{M_1, M_2 \dots M_m\}$  where  $M_i = \alpha_i \cdot P_i$ . Since each  $M_i$  is a scaled permutation matrix, the sum of any subset of matrices is a scaled double-stochastic matrix. An algorithm  $\mathcal{A}$  for *mix-net* needs to perform three operations:

1. Decompose  $M$  into the set  $P$  of permutation matrices.
2. Define the subset of matrices  $P^{(da)} \subseteq P$  that will be served using DEMAND-AWARE link scheduling. Let  $P^{(rot)} = P \setminus P^{(da)}$  be the set that will be served using ROTOR link scheduling. We define  $M^{(da)} = \sum_{P_i \in P^{(da)}} M_i$  and  $M^{(rot)} = \sum_{P_i \in P^{(rot)}} M_i$ . Note that  $M = M^{(da)} + M^{(rot)}$ .
3. Define the packet scheduling for  $M^{(da)}$  using the DEMAND-AWARE link scheduling and the packet scheduling for  $M^{(rot)}$  using the ROTOR link scheduling.

Given such an algorithm  $\mathcal{A}$ , the DCT for *mix-net* is

$$DCT_{mix}(M, \mathcal{A}) = DCT_{da}(M^{(da)}) + DCT_{rot}(M^{(rot)}). \quad (3)$$

An optimal algorithm for *mix-net* is an algorithm that minimizes the overall DCT by finding the best decomposition and best partition for  $M^{(da)}$  and  $M^{(rot)}$ . The above definition can be extended to non-saturated matrices (for  $M, M^{(da)}$  and  $M^{(rot)}$ ), for simplicity, we keep them saturated for now.

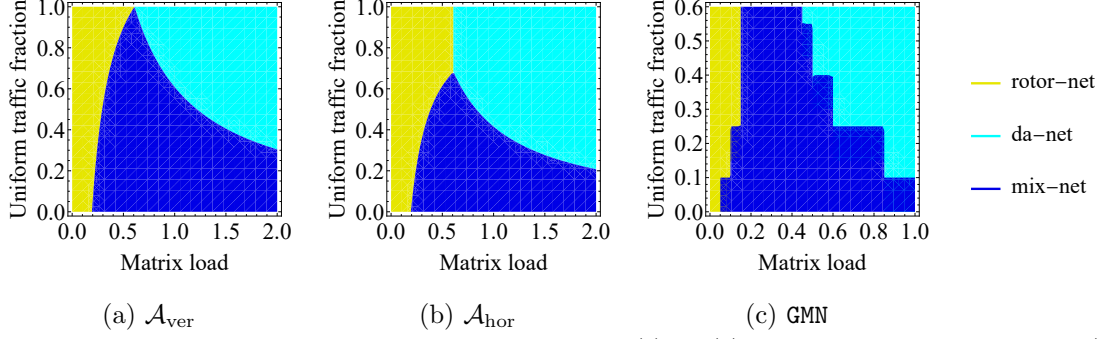


Figure 19: Comparison of the DCT of *da-net*, *rotor-net*, and *mix-net*. (a) and (b) in the multi-permutation matrix  $M(n, m, u, L)$  for  $n = 64$  and  $m = 20$ , (c) on simulation traffic. Regions denote the system with the lowest DCT.

### 5.1. A Case Study

In this section, we consider a simple case study to evaluate the DCT of the three systems.

---

#### Algorithm 3: The GreedyMixNet Algorithm

---

**Input:**  $M \in \mathbf{R}^{n \times n}$  scaled double stochastic

**Output:**  $M^{(da)}$ ,  $M^{(rot)}$ , two sub-matrices of  $M$  to schedule on either *rotor-net* or *da-net*

---

- 1 Initialize output:  $P^{(da)} = \emptyset$ ,  $P^{(rot)} = \emptyset$
  - 2 Decompose  $M$  to a set permutation matrices  $P(M)$  using maximum matchings
  - 3 **forall**  $P_i \in P(M)$  **do**
  - 4  $M_i = \alpha_i \cdot P_i$
  - 5 **if**  $DCT_{da}(M_i) \leq DCT_{rot}(M_i)$  **then**
  - 6  $P^{(da)} = P^{(da)} \cup P_i$
  - 7 **else**
  - 8  $P^{(rot)} = P^{(rot)} \cup P_i$
  - 9  $M^{(da)} = \sum_{P_i \in P^{(da)}} M_i$  and  $M^{(rot)} = \sum_{P_i \in P^{(rot)}} M_i$
- 

Let  $M(n, m, u, L)$  be an  $n \times n$  scaled doubly stochastic demand matrix where the sum of each row and column is equal to  $L$ .  $M(n, m, u, L)$  is built from a uniform matrix  $M^u$  with  $|M^u| = uLn$  and a multi-permutation matrix  $M^m$  with  $|M^m| = (1 - u)Ln$ .  $M^m$  is the union of  $m$  scaled permutation matrices, where each row and each column has  $m$  entries of  $\frac{1-u}{m}L$ , and the entries on the diagonal are zero. W.l.o.g. we assume that  $0 < u < 1$  and  $1 \leq m \leq (n - 2)$ . The matrix  $M(n, m, u, L)$  is an abstraction of the simulation traffic where a  $u$  fraction of the traffic has uniform flow sizes and a  $1 - u$  fraction of the traffic is mostly sparse with large flows, so decomposable to a few matchings. This simplified model will help us yield clear and tractable insights while remaining broadly applicable. Conceptually, the two components of our model capture bursty/high ( $m$  matchings) volume and uniform/all-to-all traffic patterns, which is a common assumption in other related papers [67, 41].

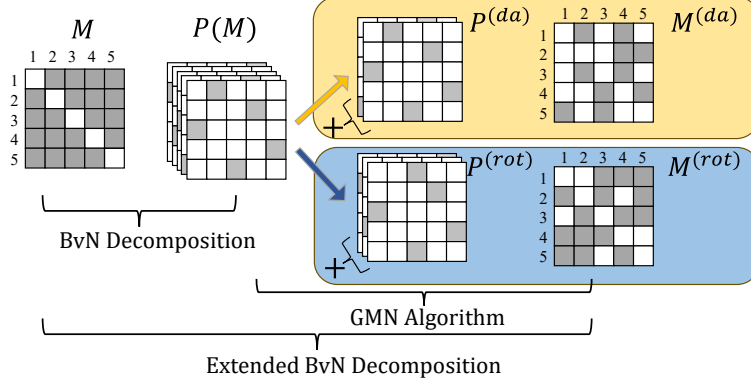


Figure 20: Schematic figure that details our extension to BvN.

In [Appendix C](#), we analytically compute the DCT of *da-net*, *rotor-net*, and two versions of *mix-net* denoted as  $\mathcal{A}_{\text{ver}}$  and  $\mathcal{A}_{\text{hor}}$  for the case study demand matrices  $M(n, m, u, L)$ . Therefore, for these demand matrices, we can systematically compare when each system, *da-net*, *rotor-net*, or *mix-net*, has the smallest DCT. Figure 19 presents such a 2-dimensional DCT map for the reconfiguration times of the simulation setup, with  $n = 64$ , and  $m = 20$  matchings. The  $Y$  axis of the map is the fraction of uniform traffic  $u$ , and the  $X$  axis is the normalized load  $\frac{L}{r}$  where  $r$  is the link rate from the simulation. Each point on this map represents a different demand matrix  $M(n, m, u, L)$ . We then divide the map into three regions, and each region shows the range of parameters for which each system *da-net*, *rotor-net*, or *mix-net* has the smallest DCT. Figure 19 (a) and (b) consider  $\mathcal{A}_{\text{ver}}$  and  $\mathcal{A}_{\text{hor}}$  for *mix-net*, respectively. We can see that for a large set of parameters *mix-net* has the smallest DCT, so better throughput than *rotor-net* and *da-net*.

## 5.2. The Greedy Algorithm

Algorithm 3 presents **GreedyMixNet** (GMN), a natural and simple algorithm to minimize  $DCT_{\text{mix}}$  for *mix-net*. Given a saturated demand matrix  $M$  as input, the algorithm first decomposes  $M$  into a set of permutation matrices  $P(M)$  and the corresponding scaling factors  $\alpha$ .

It does so in a standard way, iteratively and greedily, computing a maximum matching in  $M$ , setting  $M_i$  to be a scaled permutation matrix  $\alpha_i P_i$  (e.g., by considering  $\alpha_i$  as the minimum value in the matching), and removing  $M_i$  from  $M$ . This method is guaranteed to decompose  $M$  to a set (not necessarily minimal) of scaled permutation matrices, as proven by the BvN theorem. Next, for each  $M_i$ , we then add  $P_i$  either to  $P^{(da)}$  if  $DCT_{da}(M_i) \leq DCT_{rot}(M_i)$  and otherwise we add it to  $P^{(rot)}$ . Since  $M_i$  is a scaled permutation matrix, we can easily calculate the *optimal* DCT of each individual matching  $M_i$  on both *rotor-net* and *da-net* and determine the partition of matrices to sub-systems. For *da-net* and  $DCT_{da}$ , using Equation (1) with  $m = 1$  and for *rotor-net* and  $DCT_{rot}$  we can use Equation (2). We then define, as described earlier, the traffic that will use the DEMAND-AWARE scheduler as  $M^{(da)} = \sum_{P_i \in P^{(da)}} M_i$  and the traffic that will use the ROTOR scheduler as  $M^{(rot)} = \sum_{P_i \in P^{(rot)}} M_i$ .

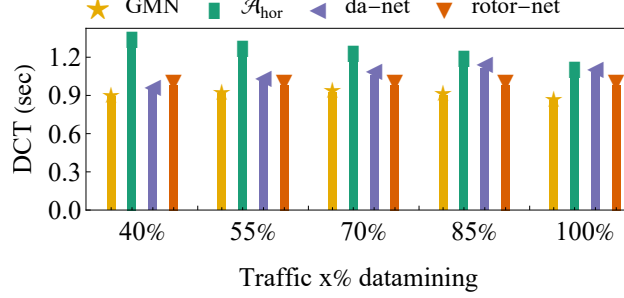


Figure 21: Empirical results on the 40% load traces showing the DCT of **GreedyMixNet**,  $\mathcal{A}_{hor}$ , *da-net* and *rotor-net*.

The  $DCT_{mix}$  of the algorithm is then defined as in Equation (3), and using Equation (1) and (2) as,

$$DCT_{mix}(M, \text{GMN}) = \frac{|M^{(da)}|}{nr} + mR_d + \frac{2}{\eta r} \frac{|M^{(rot)}|}{n}. \quad (4)$$

As described in Figure 20, we view the result of the GMN algorithm as a novel extension of the use of BvN to demand-oblivious topologies, where it was previously only used for demand-aware topologies such as in [67, 21]. GMN is using the results of the BvN decomposition to divide the matrix into two different matrices with different proprieties. Since  $M^{(rot)}$  would typically be composed of many, small flows, it represents relatively uniform traffic, while  $M^{(da)}$  composed of the largest flows represents skewed traffic. As we'll see, this application of the BvN decomposition is what allows *mix-net* to perform better.

Let us now bound the DCT of *mix-net* with GMN.

**Theorem 5.1.** *For any saturated demand matrix  $M$ ,*

$$DCT_{mix}(M, \text{GMN}) \leq \min(DCT_{rot}(M, \text{Val}), DCT_{da}(M, \text{BvN})).$$

We can examine the performance of GMN and *mix-net* on the case study demand from Section 5.1 (i.e.,  $M(n, m, u, L)$ ). For this case, we can show that GMN and *mix-net* will match the DCT of the best of the three systems we considered for  $M(n, m, u, L)$ . For example, GMN will match the results of each region in Figure 19 (a)

**Empirical Evaluation of GreedyMixNet:** We evaluate GMN on the same traffic as generated for the simulations in Section 4. The results, shown in Figure 19 (c), are similar in spirit to the theoretic results in Figure 19 (a) & (b). This demonstrates how our simplified traffic model effectively captures the essence of more realistic traffic.

Figure 21 shows an in-depth look on the 0.4 load column in Figure 19 (c). The height of each bar represents the DCT of each of the four algorithms, **GreedyMixNet**,  $\mathcal{A}_{hor}$ , *rotor-net*, and *da-net* for different DATAMINING shares. This figure shows that **GreedyMixNet** outperforms all other algorithms, consistently achieving a lower DCT of about 0.88 s. For *da-net*, the DCT increases with DATAMINING share, from about 0.94 s to 1.08 s. The DCT of *rotor-net* is 0.982 s for all DATAMINING shares since all matrices have the same

volume. While these are the main takeaways of our empirical study, we discuss Figure 19 and Figure 21 more in depth in Appendix C.

## 6. Conclusions

Motivated by the desire to maximize throughput in datacenters whose workloads evolve over time, we presented a self-adjusting datacenter network whose topology can be quickly adapted to dynamically and optimally match the workload requirements. Our architecture, D3, is enabled by Sirius and relies on a novel decentralized control plane with links and packet scheduling.

Our work opens several interesting avenues for future research. In particular, while we have primarily focused on performance, it will be interesting to study how our architecture can be extended to provide resilience under different failure models. It will also be interesting to further study scalability aspects related to the limited or heterogeneous number of ports or laser frequencies.

**Declaration of generative AI and AI-assisted technologies in the manuscript preparation process:** During the preparation of this work the author(s) used Chat-GPT, Gemini for: Polishing, improving readability and grammar, in various sections. Assistance in related work sources search. After using this tool/service, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the published article.

**Acknowledgment:** This project received funding from the Israel Science Foundation (ISF) grant no. 2497/23.

## References

- [1] Vamsi Addanki, Chen Avin, and Stefan Schmid. 2023. Mars: Near-Optimal Throughput with Shallow Buffers in Reconfigurable Datacenter Networks. *Proc. ACM Meas. Anal. Comput. Syst.* 7, 1, Article 2 (mar 2023), 43 pages. <https://doi.org/10.1145/3579312>
- [2] Mohammad Al-Fares, Alexander Loukissas, and Amin Vahdat. 2008. A scalable, commodity data center network architecture. In *ACM SIGCOMM Comput. Commun. Rev. (CCR)*, Vol. 38. ACM, 63–74.
- [3] Mohammad Alizadeh, Albert Greenberg, David A Maltz, Jitendra Padhye, Parveen Patel, Balaji Prabhakar, Sudipta Sengupta, and Murari Sridharan. 2010. Data center tcp (dctcp). In *Proceedings of the ACM SIGCOMM 2010 conference*. 63–74.
- [4] Daniel Amir, Nitika Saran, Tegan Wilson, Robert Kleinberg, Vishal Shrivastav, and Hakim Weatherspoon. 2024. Shale: A Practical, Scalable Oblivious Reconfigurable Network. In *Proceedings of the ACM SIGCOMM 2024 Conference (Sydney, NSW, Australia) (ACM SIGCOMM '24)*. 449–464.

- [5] Chen Avin, Manya Ghobadi, Chen Griner, and Stefan Schmid. 2020. On the Complexity of Traffic Traces and Implications. *Proc. ACM Meas. Anal. Comput. Syst.* 4, 1, Article 20 (may 2020), 29 pages.
- [6] Chen Avin and Stefan Schmid. 2021. ReNets: Statically-Optimal Demand-Aware Networks. In *Proc. SIAM Symposium on Algorithmic Principles of Computer Systems (APOCS)*.
- [7] Hitesh Ballani, Paolo Costa, Raphael Behrendt, Daniel Cletheroe, Istvan Haller, Krzysztof Jozwik, Fotini Karinou, Sophie Lange, Kai Shi, Benn Thomsen, et al. 2020. Sirius: A flat datacenter network with nanosecond optical switching. In *Proc. ACM SIGCOMM 2020 Conference*. 782–797.
- [8] Theophilus Benson, Aditya Akella, and David A Maltz. 2010. Network traffic characteristics of data centers in the wild. In *Proc. ACM SIGCOMM conference on Internet measurement*. ACM, 267–280.
- [9] Marcin Bienkowski, David Fuchssteiner, Jan Marcinkowski, and Stefan Schmid. 2020. Online Dynamic B-Matching with Applications to Reconfigurable Datacenter Networks. In *Proc. 38th International Symposium on Computer Performance, Modeling, Measurements and Evaluation (PERFORMANCE)*.
- [10] Garrett Birkhoff. 1946. Tres observaciones sobre el algebra lineal. *Univ. Nac. Tucuman, Ser. A* 5 (1946), 147–154.
- [11] Peirui Cao, Shizhen Zhao, Min Yee The, Yunzhuo Liu, and Xinbing Wang. 2021. TROD: Evolving from electrical data center to optical data center. In *2021 IEEE 29th International Conference on Network Protocols (ICNP)*. IEEE, 1–11.
- [12] Peirui Cao, Shizhen Zhao, Dai Zhang, Zhuotao Liu, Mingwei Xu, Min Yee Teh, Yunzhuo Liu, Xinbing Wang, and Chenghu Zhou. 2023. Threshold-based routing-topology co-design for optical data center. *IEEE/ACM Transactions on Networking* 31, 6 (2023), 2870–2885.
- [13] Cheng-Shang Chang, Wen-Jyh Chen, and Hsiang-Yi Huang. 2000. Birkhoff-von Neumann input buffered crossbar switches. In *Proceedings IEEE INFOCOM 2000. Conference on Computer Communications. Nineteenth Annual Joint Conference of the IEEE Computer and Communications Societies (Cat. No. 00CH37064)*, Vol. 3. IEEE, 1614–1623.
- [14] K. Chen, A. Singla, A. Singh, K. Ramachandran, L. Xu, Y. Zhang, X. Wen, and Y. Chen. 2014. OSA: An Optical Switching Architecture for Data Center Networks With Unprecedented Flexibility. *IEEE/ACM Transactions on Networking* 22, 2 (April 2014), 498–511.
- [15] Li Chen, Kai Chen, Zhonghua Zhu, Minlan Yu, George Porter, Chunming Qiao, and Shan Zhong. 2017. Enabling Wide-Spread Communications on Optical Fabric with Megaswitch. In *Proc. 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)* (Boston, MA, USA). USA, 577–593.

- [16] Federico De Marchi, Jialong Li, Ying Zhang, Wei Bai, and Yiting Xia. 2025. Unlocking Superior Performance in Reconfigurable Data Center Networks with Credit-Based Transport. In *Proceedings of the ACM SIGCOMM 2025 Conference*. 842–860.
- [17] M. Dinitz and B. Moseley. 2020. Scheduling for Weighted Flow and Completion Times in Reconfigurable Networks. In *IEEE Conference on Computer Communications (INFOCOM)*. 1043–1052.
- [18] Fred Douglass, Seth Robertson, Eric Van den Berg, Josephine Micallef, Marc Pucci, Alex Aiken, Maarten Hattink, Mingoo Seok, and Keren Bergman. 2021. FLEET—Fast Lanes for Expedited Execution at 10 Terabits: Program Overview. *IEEE Internet Computing* 25, 3 (2021), 79–87.
- [19] Fanny Dufossé and Bora Uçar. 2016. Notes on Birkhoff–von Neumann decomposition of doubly stochastic matrices. *Linear Algebra Appl.* 497 (2016), 108–115.
- [20] Nathan Farrington, George Porter, Sivasankar Radhakrishnan, Hamid Hajabdolali Bazzaz, Vikram Subramanya, Yeshaiah Fainman, George Papen, and Amin Vahdat. 2011. Helios: a hybrid electrical/optical switch architecture for modular data centers. *ACM SIGCOMM Comput. Commun. Rev. (CCR)* 41, 4 (2011), 339–350.
- [21] Monia Ghobadi, Ratul Mahajan, Amar Phanishayee, Nikhil Devanur, Janardhan Kulkarni, Gireeja Ranade, Pierre-Alexandre Blanche, Houman Rastegarfar, Madeleine Glick, and Daniel Kilper. 2016. Projector: Agile reconfigurable data center interconnect. In *Proc. ACM SIGCOMM 2016 Conference*. ACM, 216–229.
- [22] Albert Greenberg, James R Hamilton, Navendu Jain, Srikanth Kandula, Changhoon Kim, Parantap Lahiri, David A Maltz, Parveen Patel, and Sudipta Sengupta. 2009. VL2: a scalable and flexible data center network. In *Proc. ACM SIGCOMM 2009 Conference*. 51–62.
- [23] Chen Griner and Chen Avin. 2025. Integrating Topology and Traffic Engineering to Maximize Throughput in Reconfigurable Networks. *Computer Networks* (2025), 111803.
- [24] Chen Griner, Johannes Zerwas, Andreas Blenk, Manya Ghobadi, Stefan Schmid, and Chen Avin. 2021. Cerberus: The Power of Choices in Datacenter Topology Design (A Throughput Perspective). *Proc. ACM SIGMETRICS* (2021), 1–33.
- [25] Chuanxiong Guo, Guohan Lu, Dan Li, Haitao Wu, Xuan Zhang, Yunfeng Shi, Chen Tian, Yongguang Zhang, and Songwu Lu. 2009. BCube: a high performance, server-centric network architecture for modular data centers. *ACM SIGCOMM Computer Communication Review* 39, 4 (2009), 63–74.
- [26] Matthew Nance Hall, Klaus-Tycho Foerster, Stefan Schmid, and Ramakrishnan Durairajan. 2021. A Survey of Reconfigurable Optical Networks. *Optical Switching and Networking* 41 (2021), 100621.



- [27] Navid Hamedazimi, Zafar Qazi, Himanshu Gupta, Vyas Sekar, Samir R Das, Jon P Longtin, Himanshu Shah, and Ashish Tanwer. 2014. Firefly: A reconfigurable wireless data center fabric using free-space optics. In *ACM SIGCOMM Comput. Commun. Rev. (CCR)*, Vol. 44. ACM, 319–330.
- [28] Michelle Hampson. 2021. Reconfigurable Optical Networks Will Move Supercomputer Data 100X Faster. In *IEEE Spectrum*.
- [29] Mark Handley, Costin Raiciu, Alexandru Agache, Andrei Voinescu, Andrew W Moore, Gianni Antichi, and Marcin Wójcik. 2017. Re-architecting Datacenter Networks and Stacks for Low Latency and High Performance. In *Proc. ACM SIGCOMM 2017 Conference*. 29–42.
- [30] Jiaao He, Jidong Zhai, Tiago Antunes, Haojie Wang, Fuwen Luo, Shangfeng Shi, and Qin Li. 2022. Fastermoe: modeling and optimizing training of large-scale dynamic pre-trained models. In *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*. 120–134.
- [31] Sangeetha Abdu Jyothi, Ankit Singla, P Brighten Godfrey, and Alexandra Kolla. 2016. Measuring and understanding throughput of network topologies. In *SC’16: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 761–772.
- [32] Srikanth Kandula, Jitendra Padhye, and Paramvir Bahl. 2009. Flyways to de-congest data center networks. In *Proc. ACM Workshop on Hot Topics in Networks (HotNets)*.
- [33] Srikanth Kandula, Sudipta Sengupta, Albert Greenberg, Parveen Patel, and Ronnie Chaiken. 2009. The nature of data center traffic: measurements & analysis. In *Proc. 9th ACM SIGCOMM conference on Internet measurement*. 202–208.
- [34] Simon Kassing, Asaf Valadarsky, Gal Shahaf, Michael Schapira, and Ankit Singla. 2017. Beyond fat-trees without antennae, mirrors, and disco-balls. In *Proc. ACM SIGCOMM 2017 Conference*. 281–294.
- [35] Janardhan Kulkarni, Stefan Schmid, and Pawel Schmidt. 2021. Scheduling Opportunistic Links in Two-Tiered Reconfigurable Datacenters. In *33rd ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*.
- [36] F Thomson Leighton. 2014. *Introduction to parallel algorithms and architectures: Arrays· trees· hypercubes*. Elsevier.
- [37] Jialong Li, Haotian Gong, Federico De Marchi, Aoyu Gong, Yiming Lei, Wei Bai, and Yiting Xia. 2024. Uniform-Cost Multi-Path Routing for Reconfigurable Data Center Networks. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 433–448.

- [38] Yuliang Li, Rui Miao, Hongqiang Harry Liu, Yan Zhuang, Fei Feng, Lingbo Tang, Zheng Cao, Ming Zhang, Frank Kelly, Mohammad Alizadeh, et al. 2019. HPCC: High precision congestion control. In *Proc. ACM SIGCOMM 2019 conference*. 44–58.
- [39] Cong Liang, Xiangli Song, Jing Cheng, Mowei Wang, Yashe Liu, Zhenhua Liu, Shizhen Zhao, and Yong Cui. 2024. NegotiaToR: Towards A Simple Yet Effective On-demand Reconfigurable Datacenter Network. In *Proceedings of the ACM SIGCOMM 2024 Conference* (Sydney, NSW, Australia) (*ACM SIGCOMM '24*). 415–432.
- [40] Xudong Liao, Yijun Sun, Han Tian, Xinchun Wan, Yilun Jin, Zilong Wang, Zhenghang Ren, Xinyang Huang, Wenxue Li, Kin Fai Tse, et al. 2025. MixNet: A Runtime Reconfigurable Optical-Electrical Fabric for Distributed Mixture-of-Experts Training. In *Proceedings of the ACM SIGCOMM 2025 Conference*. 554–574.
- [41] He Liu, Matthew K Mukerjee, Conglong Li, Nicolas Feltman, George Papen, Stefan Savage, Srinivasan Seshan, Geoffrey M Voelker, David G Andersen, Michael Kaminsky, et al. 2015. Scheduling techniques for hybrid circuit/packet networks. In *Proceedings of the 11th ACM Conference on Emerging Networking Experiments and Technologies*. ACM, 41.
- [42] Hong Liu, Ryohei Urata, Kevin Yasumura, Xiang Zhou, Roy Bannan, Jill Berger, Pedram Dashti, Norm Jouppi, Cedric Lam, Sheng Li, Erji Mao, Daniel Nelson, George Papen, Mukarram Tariq, and Amin Vahdat. 2023. Lightwave Fabrics: At-Scale Optical Circuit Switching for Datacenter and Machine Learning Systems. In *Proc. ACM SIGCOMM 2023 Conference*. 499–515. <https://doi.org/10.1145/3603269.3604836>
- [43] Vincent Liu, Daniel Halperin, Arvind Krishnamurthy, and Thomas Anderson. 2013. F10: A fault-tolerant engineered network. In *Presented as part of the 10th {USENIX} Symposium on Networked Systems Design and Implementation ({NSDI} 13)*. 399–412.
- [44] William M Mellette, Rajdeep Das, Yibo Guo, Rob McGuinness, Alex C Snoeren, and George Porter. 2020. Expanding across time to deliver bandwidth efficiency and low latency. In *Proc. 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. 1–18.
- [45] William M Mellette, Alex Forencich, Rukshani Athapathu, Alex C Snoeren, George Papen, and George Porter. 2024. Realizing RotorNet: Toward Practical Microsecond Scale Optical Networking. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 392–414.
- [46] William M Mellette, Rob McGuinness, Arjun Roy, Alex Forencich, George Papen, Alex C Snoeren, and George Porter. 2017. Rotornet: A scalable, low-complexity, optical datacenter network. In *Proc. ACM SIGCOMM 2017 conference*. ACM, 267–280.

- [47] Jeffrey C. Mogul and Lucian Popa. 2012. What We Talk About when We Talk About Cloud Network Performance. *ACM SIGCOMM Comput. Commun. Rev. (CCR)* 42, 5 (Sept. 2012), 44–48.
- [48] Jeffrey C Mogul and Lucian Popa. 2012. What we talk about when we talk about cloud network performance. *ACM SIGCOMM Computer Communication Review* 42, 5 (2012), 44–48.
- [49] Pooria Namyar, Sucha Supittayapornpong, Mingyang Zhang, Minlan Yu, and Ramesh Govindan. 2021. A Throughput-Centric View of the Performance of Datacenter Topologies. In *Proc. ACM SIGCOMM Conference* (Virtual Event, USA). 349–369.
- [50] NVIDIA. 2024. *DGX-SuperPod GB200 Datasheet*. Technical Report. NVIDIA Corporation. <https://www.nvidia.com/en-us/data-center/dgx-superpod/>.
- [51] George Porter, Richard Strong, Nathan Farrington, Alex Forencich, Pang Chen-Sun, Tajana Rosing, Yeshaiah Fainman, George Papen, and Amin Vahdat. 2013. Integrating Microsecond Circuit Switching into the Data Center. In *Proc. ACM SIGCOMM 2013 Conference*. New York, NY, USA, 447–458. <https://doi.org/10.1145/2486001.2486007>
- [52] Leon Poutievski, Omid Mashayekhi, Joon Ong, Arjun Singh, Mukarram Tariq, Rui Wang, Jianan Zhang, Virginia Beauregard, Patrick Conner, Steve Gribble, et al. 2022. Jupiter evolving: transforming google’s datacenter network via optical circuit switches and software-defined networking. In *Proc. ACM SIGCOMM 2022 Conference*. 66–85.
- [53] Sundararajan Renganathan and Nick McKeown. 2025. Chronos: Prescheduled circuit switching for LLM training. In *Proceedings of the 2nd Workshop on Networks for AI Computing*. 89–97.
- [54] Arjun Roy, Hongyi Zeng, Jasmeet Bagga, George Porter, and Alex C Snoeren. 2015. Inside the social network’s (datacenter) network. In *ACM SIGCOMM Computer Communication Review*, Vol. 45. ACM, 123–137.
- [55] Sanjay Rungta, Greg Botts, Matthew Gray, Seth Gehring, and Mohammad Ali. 2025. *IT@Intel: Scaling Intel’s Data Centers with Software-Defined Networking and Automation*. Technical Report. Intel Corporation. <https://www.intel.com/content/dam/www/central-libraries/us/en/documents/2025-03/it-scaling-dc-with-sdn-paper.pdf> Published April 2025; Intel IT Best Practices.
- [56] Ken-ichi Sato. 2023. Optical switching will innovate intra data center networks [Invited Tutorial]. *Journal of Optical Communications and Networking* 16, 1 (2023), A1–A23.
- [57] Stefan Schmid, Chen Avin, Christian Scheideler, Michael Borokhovich, Bernhard Haeupler, and Zvi Lotker. 2016. Splaynet: Towards locally self-adjusting networks. *IEEE/ACM Transactions on Networking (ToN)* 24, 3 (2016), 1421–1433.

- [58] Roy Schwartz, Mohit Singh, and Sina Yazdanbod. 2019. Online and Offline Greedy Algorithms for Routing with Switching Costs. *arXiv preprint arXiv:1905.02800* (2019).
- [59] Shai Shalev-Shwartz et al. 2012. Online learning and online convex optimization. *Foundations and Trends® in Machine Learning* 4, 2 (2012), 107–194.
- [60] Arjun Singh, Joon Ong, Amit Agarwal, Glen Anderson, Ashby Armistead, Roy Bannan, Seb Boving, Gaurav Desai, Bob Felderman, Paulie Germano, et al. 2015. Jupiter rising: A decade of clos topologies and centralized control in google’s datacenter network. *ACM SIGCOMM Comput. Commun. Rev. (CCR)* 45, 4 (2015), 183–197.
- [61] Ankit Singla, Chi-Yao Hong, Lucian Popa, and Philip Brighten Godfrey. 2012. Jellyfish: Networking Data Centers, Randomly.. In *Proc. USENIX Symposium on Networked Systems Design and Implementation (NSDI 12)*, Vol. 12. 17–17.
- [62] Min Yee Teh, Zhenguo Wu, and Keren Bergman. 2020. Flexspander: augmenting expander networks in high-performance systems with optical bandwidth steering. *IEEE/OSA Journal of Optical Communications and Networking* 12, 4 (2020), B44–B54.
- [63] Min Yee Teh, Zhenguo Wu, Madeleine Glick, Sebastien Rumley, Manya Ghobadi, and Keren Bergman. 2022. Performance trade-offs in reconfigurable networks for HPC. *Journal of Optical Communications and Networking* 14, 6 (2022), 454–468.
- [64] Min Yee Teh, Shizhen Zhao, Peirui Cao, and Keren Bergman. 2022. Enabling quasi-static reconfigurable networks with robust topology engineering. *IEEE/ACM Transactions on Networking* 31, 3 (2022), 1056–1070.
- [65] Ryohei Urata, Hong Liu, Kevin Yasumura, Erji Mao, Jill Berger, Xiang Zhou, Cedric Lam, Roy Bannan, Darren Hutchinson, Daniel Nelson, et al. 2023. Apollo: Large-Scale Deployment of Optical Circuit Switching for Datacenter Networking. In *Optical Fiber Communication Conference*. Optica Publishing Group, 1–3.
- [66] Leslie G. Valiant. 1982. A scheme for fast parallel communication. *SIAM journal on computing* 11, 2 (1982), 350–361.
- [67] Shaileshh Bojja Venkatakrishnan, Mohammad Alizadeh, and Pramod Viswanath. 2018. Costly circuits, submodular schedules and approximate Carathéodory Theorems. *Queueing Systems* 88, 3-4 (2018), 311–347.

- [68] Guohui Wang, David G Andersen, Michael Kaminsky, Michael Kozuch, TS Ng, Konstantina Papagiannaki, Madeleine Glick, and Lily Mummert. 2009. Your data center is a router: The case for reconfigurable optical circuit switched paths. *Proc. ACM Hotnets VIII* (2009).
- [69] Guohui Wang, David G Andersen, Michael Kaminsky, Konstantina Papagiannaki, TS Ng, Michael Kozuch, and Michael Ryan. 2011. c-Through: Part-time optics in data centers. *ACM SIGCOMM Comput. Commun. Rev. (CCR)* 41, 4 (2011), 327–338.
- [70] Weiyang Wang, Moein Khazraee, Zhizhen Zhong, Manya Ghobadi, Zhihao Jia, Dheevatsa Mudigere, Ying Zhang, and Anthony Kewitsch. 2023. {TopoOpt}: Co-optimizing network topology and parallelization strategy for distributed training jobs. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. 739–767.
- [71] Haitao Wu, Guohan Lu, Dan Li, Chuanxiong Guo, and Yongguang Zhang. 2009. MDCube: a high performance network structure for modular data center interconnection. In *Proceedings of the 5th international conference on Emerging networking experiments and technologies*. ACM, 25–36.
- [72] Zhenguo Wu, Benjamin Klenk, Larry Dennison, and Keren Bergman. 2025. ACTINA: Adapting Circuit-Switching Techniques for AI Networking Architectures. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1211–1222.
- [73] Xuwei Xue and Nicola Calabretta. 2022. Nanosecond optical switching and control system for data center networks. *Nature communications* 13, 1 (2022), 2257.
- [74] Johannes Zerwas, Csaba Györgyi, Andreas Blenk, Stefan Schmid, and Chen Avin. 2023. Duo: A high-throughput reconfigurable datacenter network using local routing and control. *Proc. ACM Meas. Anal. Comput. Syst.* 7, 1 (2023), 1–25.
- [75] Mingyang Zhang, Jianan Zhang, Rui Wang, Ramesh Govindan, Jeffrey C Mogul, and Amin Vahdat. 2021. Gemini: Practical Reconfigurable Datacenter Networks with Topology and Traffic Engineering. *arXiv preprint arXiv:2110.08374* (2021). arXiv:2110.08374 [cs.NI]
- [76] Qiao Zhang, Vincent Liu, Hongyi Zeng, and Arvind Krishnamurthy. 2017. High-resolution Measurement of Data Center Microbursts. In *Proc. 2017 Internet Measurement Conference (London, United Kingdom) (IMC '17)*. 78–85. <https://doi.org/10.1145/3131365.3131375>
- [77] Shizhen Zhao, Peirui Cao, and Xinbing Wang. 2021. Understanding the performance guarantee of physical topology design for optical circuit switched data centers. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 5, 3 (2021), 1–24.

- [78] Xia Zhou, Zengbin Zhang, Yibo Zhu, Yubo Li, Saipriya Kumar, Amin Vahdat, Ben Y Zhao, and Haitao Zheng. 2012. Mirror mirror on the ceiling: Flexible wireless links for data centers. *ACM SIGCOMM Comput. Commun. Rev. (CCR)* 42, 4 (2012), 443–454.
- [79] Zhaoxing Zhou, Huaxi Gu, Xiaoshan Yu, Qian Zhang, and Yunhao Wang. 2025. Halo: An Efficient and Scalable Distributed Control Strategy for AWGR-Based Optical Networks. *IEEE Transactions on Networking* (2025).
- [80] Shihong Zou, Xitao Wen, Kai Chen, Shan Huang, Yan Chen, Yongqiang Liu, Yong Xia, and Chengchen Hu. 2014. VirtualKnotter: Online virtual machine shuffling for congestion resolving in virtualized datacenter. *Computer Networks* 67 (2014), 141–153.

| Variable                          | Description                                                                   |
|-----------------------------------|-------------------------------------------------------------------------------|
| $n$                               | Total number of ToRs.                                                         |
| $k$                               | Total number of spines                                                        |
| $k_r$                             | Total number of rotor spines                                                  |
| $T_i$                             | ToR $i$ , $1 \leq i \leq n$                                                   |
| $R_i$                             | Rotor $i$ , $1 \leq i \leq k_r$                                               |
| $h = n \cdot k$                   | Total number of hosts                                                         |
| $C = r \cdot \delta$              | Slot active capacity (volume to send)                                         |
| $r$                               | Link rate                                                                     |
| $\bar{C} = \frac{C \cdot k_r}{k}$ | host ToR rotor capacity                                                       |
| $h_{i,j}$                         | Host $i$ on ToR $j$ (can be translated to an integer for indexing)            |
| $T_{u,j}$                         | ToR connected on Rotor uplink $u$ on ToR $j$                                  |
| <b>Input</b>                      |                                                                               |
| $D_{i,j}^L$                       | current local demand from host $i$ to host $j$                                |
| $D_{i,j}^N$                       | current non-local demand from host $i$ to host $j$                            |
| <b>Output</b>                     |                                                                               |
| $S_{i,j}^{ld}$                    | direct local volume to send from host $i$ to $j$                              |
| $S_{i,j}^n$                       | non-local volume to send from host $i$ to $j$                                 |
| $S_{i,j,k}^{li}$                  | indirect local volume to send from host $i$ to $j$ with final destination $k$ |

Table A.3: Notation.

## Appendix A. Detailed Packet Scheduling Algorithm of D3

This section provides a more detailed description of the algorithms underlying D3's packet scheduling. First, Table A.3 summarizes the used notation. In particular, it defines the input (sizes of the local and non-local buffers) and output data structures (allocated volumes from local and non-local to the destination and from local to an intermediate host). Algorithm 5 is a sub-routine used by LLB. The routine is summarized in pseudo-code from prior work (the code base) from OPERA [44]. Given a matrix of demand to be sent, it applies a 2-dimensional fair share over the sending and receiving capacities (given as vectors). Therefore, it repeats sweeping the rows (l. 5f) and then the columns (l. 7f) until the output values converged.

Algorithm 4 is the full packet scheduling algorithm with the major steps as described in Section 3.3. The algorithm takes a per ToR perspective (the algorithm runs on ToR  $t$ ); all data structures are rack-local but the algorithm listing partially uses global indices to simplify notation.

In lines 2-6, the algorithm allocates the second hop non-local traffic and local traffic that can be sent directly to the destination. For each buffer type, it iterates over the ROTOR ports. Per port, collects the demand information (local or non-local, respectively) of all source and destination hosts that are connected by the port. Specifically, for all hosts in rack  $t$  and in the rack that is connected via ROTOR link  $b$  ( $T_{b,t}$ ). Then, it allocates the demand using the 2-dimensional fairshare (FS2D, l. 5) and stores the allocation in the respective output data structure.

The second part (lines 7- 21) allocates traffic to be sent via an intermediate host. Again, the algorithm

---

**Algorithm 4:** D3 Packet-Scheduling (LocalLB) for ToR  $t$ 


---

**Input:** Per s-d pair local  $D^l$  and non-local  $D^n$  demand

**Output:** Per s-d pair traffic to allocate for direct transmission ( $S^{ld}$ ,  $S^n$ ) and for transmission via an intermediate host ( $S^{li}$ ).

```

1 Initialize  $C_i^{TX} \leftarrow \bar{C}$ ,  $C_{i,b}^{RX} \leftarrow \frac{C}{k}$ ,  $\forall i = 1 \dots k, \forall b = 1 \dots k_r$ 
   // Allocate Non-local (n) on the 2nd hop and local (1)
2 for  $x \in \{n, l\}$  do
3   for  $b = 1 \dots k_r$  do
4      $W_{i,j} \leftarrow D_{h_{i,t}, h_{j,T_{b,t}}}^x \quad \forall i = 1 \dots k, j = 1 \dots k$ 
5      $S^x \leftarrow FS2D(W, C^{TX}, C^{RX})$ 
6     Update  $C^{RX}, C^{TX}, D^x$ 
   // Allocate new indirect traffic on the remaining capacity
7 for  $b = 1 \dots k_r$  do
   // Iterate over all hosts in ToR connected via  $b$ 
8   for  $\forall a \in \{h_{j,T_{b,t}}, j = 1 \dots k\}$ , order RR over slots do
9     // Find the largest demand per source host capped by remaining sending capacity
10     $v_{i,m} \leftarrow D_{h_{i,t}, h_m}^l \quad \forall i = 1 \dots k, m = 1 \dots nk$ 
11     $v_{i,m} = v_{i,m} - \frac{C}{k}$  and zero all  $< 0$ 
12     $y_i \leftarrow \arg \max_m v_{i,m}, \forall i = 1 \dots k$ 
13     $z_i \leftarrow \min\{v_{i,y_i}, C_i^{TX}\}$ 
14     $n_z \leftarrow \text{number of } z_i > 0$ 
15    for  $\forall i$  where  $z_i > 0$  in non-decreasing order of  $z_i$  do
16      if  $z_i > C_{a,b}^{RX}$  then
17        Allocate  $\frac{C_{a,b}^{RX}}{n_z}$  for all  $i$  with  $z_i > 0$ 
18        Update  $C^{TX}, C^{RX}, D^l$ 
19        Break
20      else
21        Allocate  $z_i$  for  $i$  and update  $C^{TX}, C^{RX}, D^l$ 
22        Decrement  $n_z$ 

```

---



---

**Algorithm 5:** 2-dimensional fair share (based on code from OPERA’s repository [44]).

---

```

1  $I \in \mathbf{R}^{N \times M}, c0 \in \mathbf{R}^N, c1 \in \mathbf{R}^M$ 
2 Initialize output:  $O \in \mathbf{R}^{N \times M}$ 
3 Count elements  $> 0$  in  $I$  as  $n^+$ 
4 while  $n^+ > 0$  and max iterations not reached do
5   for  $i = 1 \dots N$  do
6      $t[i] =$  1-dimensional fair share on  $I[i]$  with capacity  $c0[i] - \sum_{j=1 \dots M} O[i, j]$ 
7   for  $j = 1 \dots M$  do
8      $t2 =$  1-dimensional fair share on  $j$ -th column of  $t$  with capacity  $c1[j] - \sum_{i=1 \dots N} O[i, j]$ 
9   Update  $t$  with  $t2$ 
10  Update  $I, O$  with  $t$ 
11  Zero out rows  $I[i]$  where  $c0[i] = 0$ 
12  Zero out columns  $I[j]$  where  $c1[j] = 0$ 
13  Count elements  $> 0$  in  $I$  as  $n^+$ 

```

---

iterates over the ROTOR ports. Per port, it iterates over the hosts in the destination rack connected via the ROTOR link ( $T_{b,t}$ ; with the order changing in a round-robin fashion between calls of the packet scheduling). In every iteration, it searches for the destination host with the highest demand per source host in  $t$  (lines 9-13). In detail, the local demands between hosts in  $t$  and all other hosts (not in  $t$ ) are gathered subtracted by the equal-share capacity before getting the largest demand per source (lines 9-11). These found values are then upper capped by the remaining sending capacity per host (l. 12) Then, it iterates over these demands in non-decreasing order. If a demand  $z_i$  exceeds the remaining capacity in the slot to the intermediate host, the equal-share of the remaining capacity is given to all remaining demands  $> 0$  (lines 15-18). If the demand does not exceed the remaining capacity of the intermediate host, it is greedily allocated (l. 20f).

## Appendix B. Additional material for Empirical Results

| Type         | Par.       | 10 G                 |                       |         | 50 G                  |                           |            | 100 G                 |                           |            |
|--------------|------------|----------------------|-----------------------|---------|-----------------------|---------------------------|------------|-----------------------|---------------------------|------------|
|              |            | Opera                | D3                    | Duo     | Opera                 | D3                        | Duo        | Opera                 | D3                        | Duo        |
| ROTOR        | $R_r$      | $10\mu s + 1.7\mu s$ | $100ns + 1.7\mu s$    | NA      | $10\mu s + 0.74\mu s$ | $100ns + 0.74\mu s$       | NA         | $10\mu s + 0.62\mu s$ | $100ns + 0.62\mu s$       | NA         |
|              | $\delta$   | $33.46 \cdot R_r$    | $54.56 \cdot R_r$     | NA      | $13.65 \cdot R_r$     | $14.65 \cdot R_r$         | NA         | $10.93 \cdot R_r$     | $12.88 \cdot R_r$         | NA         |
|              | Duty cycle | 97%                  | 98%                   | NA      | 93%                   | 93%                       | NA         | 92%                   | 92%                       | NA         |
| DEMAND-AWARE | $R_d$      | NA                   | $1ms = 10k \cdot R_r$ | $1ms$   | NA                    | $100\mu s = 2k \cdot R_r$ | $100\mu s$ | NA                    | $100\mu s = 1k \cdot R_r$ | $100\mu s$ |
|              | $\delta_d$ | NA                   | $49R_d$               | $49R_d$ | NA                    | $49R_d$                   | $49R_d$    | NA                    | $49R_d$                   | $49R_d$    |
|              | Duty cycle | NA                   | 98%                   | 98%     | NA                    | 98%                       | 98%        | NA                    | 98%                       | 98%        |

Table B.4: Reconfiguration parameters.

## Appendix C. Analysis of Case Study

We first consider the DCTs of *rotor-net* and *da-net*. For *rotor-net*, we can use Equation (2), so

$$DCT_{rot}(M(n, m, u, L)) = 2 \frac{|M(n, m, u, L)|}{\eta nr} = \frac{2L}{\eta r}. \quad (C.1)$$

For *da-net*, we can decompose the matrix into the  $m$  original permutation matrices and additionally a set of  $(n-1-m)$  permutation matrices for the rest of the uniform matrix. In total, we have  $n-1$  matchings. Using Equation (1), we get the following result,

$$DCT_{da}(M(n, m, u, L)) = \frac{L}{r} + (n-1)R_d. \quad (C.2)$$

In both cases, we note that the DCT is a function of the the sum of a row  $L$ .

Next, we study *mix-net*. For this, we need first to find a partition of  $M(n, m, u, L)$  into  $M^{(da)}$  and  $M^{(rot)}$ . We consider two potential algorithms:  $\mathcal{A}_{hor}$  and  $\mathcal{A}_{ver}$ .

The  $\mathcal{A}_{hor}$  algorithm (horizontal) is motivated by the simulation setup and sets  $M^{(da)} = M^m$  and  $M^{(rot)} = M^u$ . That is, it is an algorithm where all uniform traffic is sent with the ROTOR link scheduler, and all the permutation matrices are sent with the DEMAND-AWARE scheduler. The  $DCT_{mix}$  in this case is

$$DCT_{mix}(M(n, m, u, L), \mathcal{A}_{hor}) = DCT_{da}(M^m) + DCT_{rot}(M^u) = (1-u)\frac{L}{r} + mR_d + u\frac{2L}{\eta r}. \quad (C.3)$$

For the  $\mathcal{A}_{ver}$  algorithm (vertical), we again decompose  $M(n, m, u, L)$  into a set of  $m$  matchings sent with the DEMAND-AWARE scheduler, but here the total traffic in the  $m$  matchings also includes traffic from the uniform matrix; the remaining matrix is sent with the ROTOR scheduler. The DCT is determined by the average row size of  $M^{(da)}$  and  $M^{(rot)}$ . In  $M^{(da)}$ , a cell from one of the  $m$  matchings, has  $L\frac{1-u}{m}$  bits from  $M^m$  and  $L\frac{u}{n-1}$  bits from  $M^u$ . For the remaining entries, each of the  $(n-1-m)$  non-zero entries has  $L\frac{u}{n-1}$  bits. The  $DCT_{mix}$  is then

$$DCT_{mix}(M(n, m, u, L), \mathcal{A}_{ver}) = \frac{Lm}{r} \left( \frac{1-u}{m} + \frac{u}{n-1} \right) + mR_d + \frac{2Lu}{\eta r} \left( \frac{n-1-m}{n-1} \right). \quad (C.4)$$

### Appendix C.1. Discussion of Figures 19 and 21

In this section, we will discuss Figures 19 and 21, adding to the original discussion in the main body of the paper.

**Figure 19:** There are some interesting observations when comparing the Figures 19 (a) & (b). The blue fraction representing the vertical,  $\mathcal{A}_{ver}$ , algorithm is larger in (a), suggesting that, despite that both algorithms send the *same* size  $m$  of the permutation set,  $\mathcal{A}_{ver}$  is more efficient. A second observation is to notice the point on the  $X$  axis where  $DCT_{da} = DCT_{rot}$ . Solving for  $x$  (by Equations (C.1) and (C.2)) where  $x = \frac{L}{r}$  we get  $x = \frac{\eta(n-1)R_d}{2-\eta}$  which for our parameters is approximately  $x \approx 0.6$ . Above  $x$ , at least some

matchings will use the DEMAND-AWARE scheduler, and below  $x$ , at least some matchings will be sent to the ROTOR scheduler. This value for  $x$  gives us the bound between the *da-net* and *rotor-net* dominated areas. To generate the three different areas in the figure, we also need to find where *mix-net* is dominant (i.e., its DCT is smaller). Therefore, when,  $x \geq 0.6$  (roughly), we need to find where *mix-net* is dominant over *da-net*; that is where the value given by Equation C.2 is larger than the DCT value given by Equation C.4 for Figure 19 (a) and Equation C.3 for Figure 19 (b). When  $x < 0.6$  we need to find where *mix-net* is dominant over *rotor-net* that is, where the DCT value given in Equation C.1 is larger than the DCT value given by Equation C.4 for figure Figure 19 (a) and Equation C.3 for Figure 19 (b). Overall, we can see, as expected, that increasing the load will reach a point where *da-net* is the best system (since the longer reconfiguration time is amortized), and decreasing the load will reach a point where *rotor-net* has a smaller DCT. Looking at the Y axis, we find that with a lower  $u$  we observe higher success for *mix-net*. Lower  $u$  means a higher fraction is represented by the same 20 matchings. This makes the ROTOR scheduler less effective on them and the DEMAND-AWARE scheduler less effective on the uniform traffic, justifying the traffic partition of *mix-net*. For Figure 19 (c), Recall that GreedyMixNet needs to decompose  $M$ . In our implementation, we use a simple greedy decomposition algorithm based on BVN. Given a demand matrix  $M$ , we search for the maximal weighted matching and remove it from  $M$ . In the general case, we would fully decompose a matrix, however, since each subsequent matching found gets smaller, we decompose the matrix until there is some fraction of traffic left. While we do not explicitly consider this, we assume small traffic can be sent using the STATIC sub-topology. Let us denote this fraction as  $L_s$ . In this section, we consider  $L_s = 0.05$ , i.e. 5% of the total traffic is assumed to be sent using the STATIC sub-topology. Since the simulation traffic does not necessarily adhere to the saturation requirement, we cannot assume that  $M$  is perfectly saturated. Each matching might also not be saturated. We, therefore, use a cutoff value for the matching. As a simple heuristic, we choose the average value over their original matching as the cutoff. Each value in the found matching will be, at most, the average value after the cutoff. In Figure 19 (c), we show results based on five real traffic traces from the simulation as described earlier, in Section 4.1. The specific traces used are with a load  $L = 40\%$  with five different datamining shares (40%, 55%, 70%, 85%, 100%). Furthermore, for the sake of consistency between different loads, we normalize all matrices to the global mean matrix value. The traffic trace itself is a sequence of ToR-level traffic matrices, each representing a total of 0.05 seconds of accumulated remaining traffic from the simulation. That is, after each 0.05 seconds, we gather all pending traffic into an accumulated matrix. Each row represents a different datamining load, and each column represents an accumulated trace. That is, at the cell (0.3, 0.7), we gather the first 14 matrices from the 70% trace. To color the cells, we denote cells with less than a fraction of  $\frac{1}{12}$  of the total DCT in *rotor-net* pure *da-net* and vice versa. Otherwise, we assume the optimal system is *mix-net*.

**Figure 21:** presents the DCT results for four algorithms, GreedyMixNet,  $\mathcal{A}_{\text{hor}}$ , *da-net*, and *rotor-net*

on simulated traces; the traffic is the same as in Figure 19 (c) for a load of 40%. Note that we could only implement  $\mathcal{A}_{\text{hor}}$  and not  $\mathcal{A}_{\text{ver}}$  on these traces. Recall that traffic is generated such that a fraction of the load is from uniform traffic, and a portion is sampled from datamining. To implement  $\mathcal{A}_{\text{hor}}$ , we sent the uniform traffic to *rotor-net* and the rest to *da-net*. To implement  $\mathcal{A}_{\text{ver}}$ , we would need to know in advance how to decompose the matrix into a set of  $m$  ‘large’ matchings, for *da-net*, and a set of  $n - 1 - m$  ‘small’ matchings, for *rotor-net*, which is not normally possible. This is different than our **GreedyMixNet**, which tries to find an optimal set  $m$  matching to send to *da-net*.

Regarding the results for *rotor-net*, we see that all the results are the same for *rotor-net*. We can explain this by looking at the DCT of *rotor-net*. In Equation 2, we see that it is a function of the total load and not a function of the structure of the matrix itself. The *total* traffic of the simulated matrices was normalized to the same expected size, that is  $t \cdot L \cdot r \cdot n \cdot k$ , where  $t$  is the time represented by the matrix in seconds, resulting in a similar outcome. For the other algorithms, the BVN decomposition results in a different DCT due to the different sizes of the decomposition.

## Appendix C.2. Formal results

Formally for  $M = M(n, m, u, L)$ , we can show the following:

**Claim 1.** *Let  $M$ , a demand matrix be  $M = M(n, m, u, L)$ , then the following holds for the algorithm **GMN**.*

$$DCT_{\text{mix}}(M, \mathbf{GMN}) = \min\{DCT_{\text{mix}}(M, \mathcal{A}_{\text{ver}}), DCT_{\text{da}}(M, \mathbf{BvN}), DCT_{\text{rot}}(M, \mathbf{Val})\}. \quad (\text{C.5})$$

*Proof.* After decomposing  $M$  using **BvN**, we first observe that there are only two sizes of permutation matrices in the decomposition. The first has a size of  $\frac{uL}{n-1}$  per row. These permutations were made directly from the matrix  $M^u$ . And another set of permutations made from the overlap between  $M^u$  and  $M^m$ , these will have a row size of  $\frac{(1-u)L}{m(n-1)} + \frac{uL}{n-1}$ . Since **GMN** tests each permutation on an individual basis, it has exactly three options, namely, send all permutations to *da-net*, send all permutations to *rotor-net*, or send the smaller of the two types to *rotor-net* and the larger to *da-net*. Since the second set with the size of  $\frac{(1-u)L}{m(n-1)} + \frac{uL}{n-1}$  is always larger **GMN** will be emulating  $\mathcal{A}_{\text{ver}}$ . Lastly, note that **GMN** will never send the smaller permutations to *da-net* and the larger to *rotor-net*. This is true for any matrix, as it is a result of the DCT of both networks being linear.  $\square$

The following is a simple observation of Claim 1.

**Observation 1.** *Let  $P \in P_\pi$  be a permutation matrix. The DCT of mix-net is equal the minimal DCT of  $P$  for either *da-net* or *rotor-net*,*

$$DCT_{\text{mix}}(P, \mathbf{GMN}) = \min(DCT_{\text{rot}}(P, \mathbf{Val}), DCT_{\text{da}}(P, \mathbf{BvN})). \quad (\text{C.6})$$

Note that  $P$  is a special case in  $M(n, m, u, L)$ . The observation stems from the fact that **GMN** can only schedule the entire matrix  $P$  on either *mix-net* or *rotor-net*, since no further decomposition to permutations will be made, and that the **GMN** algorithm will schedule  $P$  where the DCT is lowest.

Now, we prove Theorem 5.1, which holds for any saturated demand matrix.

**Theorem Appendix C.1.** *For any saturated demand matrix  $M$ ,*

$$DCT_{mix}(M, \mathbf{GMN}) \leq \min(DCT_{rot}(M, \mathbf{val}), DCT_{da}(M, \mathbf{BvM})) \quad (\text{C.7})$$

*That is, the DCT of mix-net will be equal or less than the DCT of either rotor-net or da-net.*

*Proof of Theorem 5.1.* First, let us denote the set of matchings sent on *da-net* as  $M^{(da)} = \{M_i : DCT_{da}(M_i) \leq DCT_{rot}(M_i)\}$ . And let  $M^{(rot)} = \{M_i : DCT_{rot}(M_i) \leq DCT_{da}(M_i)\}$  be the set of matchings sent on *rotor-net*. Since for an individual permutation matrix *mix-net* has lower or equal DCT to either *da-net* or *rotor-net* as from Observation 1, and since both functions describing the DCT either system (in Equation (1) and Equation (2)) are linear in the total load of either  $M^{(da)}$  or  $M^{(rot)}$ , the theorem follows.  $\square$